

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

APLICACIÓN DE GRAFOS Y ÁRBOLES EN UN JUEGO

PROYECTO 2 BIM

**PROYECTO DEL SEGUNDO BIMESTRE DE LA ASIGNATURA DE
ESTRUCTURA DE DATOS Y ALGORITMOS I.**

EDISON RODRIGO SOTO PÉREZ

(edison.soto@epn.edu.ec)

ALEX ISRAEL POGO RAMIREZ

(alex.pogo@epn.edu.ec)

MANOLO ALEXANDER GUERRERO TIPÁN

(manolo.guerrero@epn.edu.ec)

DOCENTE: ING.BORIS ALFONSO ASTUDILLO ESPINOZA M.Sc.

(boris.astudillo@epn.edu.ec)

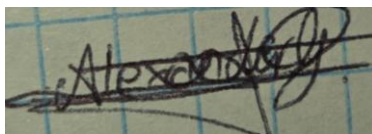
Quito, enero de 2026

CERTIFICACIONES

Nosotros, SOTO PÉREZ EDISON RODRIGO, MANOLO ALEXANDER GUERRERO TIPÁN y POGO RAMIREZ ALEX ISRAEL declaramos que el proyecto aquí descrito es de nuestra autoría; que no ha sido previamente presentado para ninguna otra asignatura o calificación a nivel individual en otras asignaturas; y, que hemos consultado las referencias bibliográficas que se incluyen en este documento.



**EDISON RODRIGO SOTO
PÉREZ**



MANOLO ALEXANDER GUERRERO TIPÁN



ALEX ISRAEL POGO RAMIREZ

Certifico que el presente proyecto del segundo bimestre fue desarrollado por SOTO PÉREZ EDISON RODRIGO, MANOLO ALEXANDER GUERRERO TIPÁN y POGO RAMIREZ ALEX ISRAEL, bajo mi supervisión.

**ING.BORIS ALFONSO ASTUDILLO ESPINOZA
M.Sc.**

DECLARACIÓN DE AUTORÍA

A través de la presente declaración, afirmamos que el proyecto aquí descrito, así como el (los) producto(s) resultante(s) del mismo, son públicos y estarán a disposición de la comunidad a través del repositorio institucional de la Escuela Politécnica Nacional; sin embargo, la titularidad de los derechos patrimoniales nos corresponde a los autores que hemos contribuido en el desarrollo del presente trabajo; observando para el efecto las disposiciones establecidas por el órgano competente en propiedad intelectual, la normativa interna y demás normas.

EDISON RODRIGO SOTO PÉREZ

MANOLO ALEXANDER GUERRERO TIPÁN

ALEX ISRAEL POGO RAMIREZ

ING.BORIS ALFONSO ASTUDILLO ESPINOZA M.Sc.

ÍNDICE DE CONTENIDO

CERTIFICACIONES	I
DECLARACIÓN DE AUTORÍA	II
ÍNDICE DE CONTENIDO.....	III
RESUMEN	IV
ABSTRACT.....	V
1 INTRODUCCIÓN.....	¡Error! Marcador no definido.
1.1 Objetivo general.....	1
1.2 Objetivos específicos	1
1.3 Alcance	1
1.4 Marco teórico	¡Error! Marcador no definido.
2 METODOLOGÍA.....	¡Error! Marcador no definido.
3 RESULTADOS, CONCLUSIONES Y RECOMENDACIONES	¡Error! Marcador no definido.
3.1 Resultados.....	¡Error! Marcador no definido.
3.2 Conclusiones	¡Error! Marcador no definido.
3.3 Recomendaciones	¡Error! Marcador no definido.
4 REFERENCIAS BIBLIOGRÁFICAS.....	¡Error! Marcador no definido.
5 ANEXOS	¡Error! Marcador no definido.
ANEXO I	¡Error! Marcador no definido.

RESUMEN

El presente proyecto tuvo como objetivo el desarrollo de un videojuego en el cual pueden participar dos jugadores. Su implementación se realizó mediante el uso de estructuras de datos, enfocándose principalmente en grafos y árboles, con el fin de optimizar la organización y gestión de la información dentro del sistema. Para su desarrollo se utilizó el lenguaje de programación **Python**, aprovechando su flexibilidad y facilidad de integración con distintas herramientas. Asimismo, se empleó **Unity** como motor gráfico, el cual fue elegido por su simplicidad y capacidad para crear un entorno visual interactivo e intuitivo para el usuario. Además, se implementaron archivos en formato **JSON** para el almacenamiento y transferencia de datos de manera estructurada, legible y eficiente. Esto facilitó tanto la comunicación entre los distintos módulos del programa como la comprensión de la información por parte de los desarrolladores. La combinación de estas tecnologías permitió la creación de un sistema funcional, escalable y mantenible, demostrando la aplicación práctica de conceptos teóricos de estructuras de datos en el desarrollo de videojuegos multijugador.

PALABRAS CLAVE: Python, Unity, JSON, grafos, árboles.

ABSTRACT

The aim of this project was the development of a video game in which two players can participate. Its implementation was carried out through the use of data structures, focusing primarily on graphs and trees, in order to optimize the organization and management of information within the system. The Python programming language was used for its development, taking advantage of its flexibility and ease of integration with various tools. Additionally, Unity was employed as the graphics engine, which was chosen for its simplicity and its ability to create an interactive and intuitive visual environment for the user. Furthermore, JSON files were implemented for the structured, readable, and efficient storage and transfer of data. This facilitated both communication between the different modules of the program and the understanding of information by the developers. The combination of these technologies enabled the creation of a functional, scalable, and maintainable system, demonstrating the practical application of theoretical data structure concepts in multiplayer video game development.

KEYWORDS: Python, Unity, JSON, graphs, trees.

1 INTRODUCCIÓN Y GENERALIDADES DEL PROYECTO

1.1 Descripción del Proyecto

El proyecto consistió en el desarrollo de un videojuego para dos jugadores utilizando estructuras de datos como grafos y árboles. El funcionamiento del juego se basa en distintos escenarios, los cuales son representados como nodos dentro de un grafo, permitiendo modelar las conexiones y transiciones entre ellos. Asimismo, se implementó un sistema de toma de decisiones en el que al jugador se le presentan diversas opciones en pantalla. Dichas decisiones se relacionan con la estructura de árboles, ya que permiten organizar y gestionar las distintas rutas y resultados posibles dentro del juego.

1.2 Objetivo general

Desarrollar un videojuego con interfaz gráfica mediante Unity que utilice estructuras de datos como grafos y árboles, con el fin de aplicar y comprender su funcionamiento en un entorno práctico de programación y en la organización eficiente de información dentro de un sistema interactivo.

1.3 Objetivos específico

1. Aprender el funcionamiento de Unity para el desarrollo de la interfaz gráfica
2. Comprender el uso de grafos y árboles
3. Implementar la lógica de programación usando Python como lenguaje
4. Implementar archivos JSON para almacenar y transferir datos de manera estructurada entre los distintos módulos del juego.

1.4 Alcance del proyecto

El proyecto abarca el desarrollo de un videojuego para dos jugadores que aplica estructuras de datos como grafos y árboles para modelar los escenarios del juego y el sistema de toma de decisiones. Los grafos permiten representar las conexiones y transiciones entre los distintos escenarios, mientras que los árboles organizan las posibles rutas y resultados derivados de las decisiones del jugador durante la partida. Asimismo, el desarrollo incluye la implementación de la lógica del juego utilizando el lenguaje de programación Python, la creación de una interfaz gráfica mediante Unity y el uso de archivos en formato JSON para el almacenamiento y la transferencia de datos. El proyecto se limita a la creación de un prototipo funcional con fines académicos, sin

contemplar características avanzadas como multijugador en línea, inteligencia artificial compleja u optimización a nivel comercial.

2 MARCO TEÓRICO

2.1 Teoría de Grafos

Definición y Conceptos:

Un GRAFO es una estructura de datos matemática que consta de un conjunto de vértices (también llamados nodos) y un conjunto de aristas (también llamadas bordes) que conectan estos vértices. [1]

En el ámbito de la informática, los grafos son ampliamente utilizados para modelar problemas relacionados con redes, rutas, conexiones y estructuras jerárquicas, debido a su flexibilidad y capacidad de representación.

Como ejemplo de su aplicación, “un grafo podría representar una red social, donde los nodos son usuarios y las aristas denotan amistades o conexiones. De esta forma, se pueden recomendar amigos o contenido a los usuarios basándose en el grafo” [1].

Tipología de Grafos:

- **Grafo Bidireccional**

Un grafo bidireccional o no dirigido es una estructura matemática utilizada en teoría de grafos, donde las aristas (o conexiones) entre nodos (o vértices) permiten el recorrido en ambas direcciones. Esto significa que, si hay una arista que conecta dos nodos A y B, se puede ir de A a B y de B a A sin restricciones. Esta característica lo distingue de los grafos dirigidos, donde las aristas tienen una dirección específica. Los grafos bidireccionales son fundamentales para modelar relaciones [2].

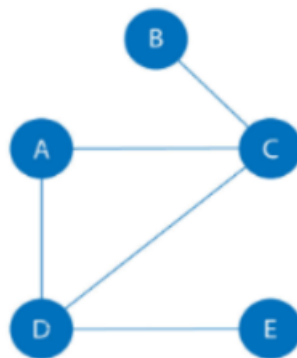


Figura 2.1.1 Gráfica de grafo no dirigido

- **Grafo dirigido**

Son también conocidos como digrafos, estos grafos tienen aristas con una dirección definida. Es como una calle de sentido único: podemos ir de A a B, pero no necesariamente de B a A. Un ejemplo perfecto es Twitter, donde puedes seguir a alguien sin que esa persona te siga a ti [3].

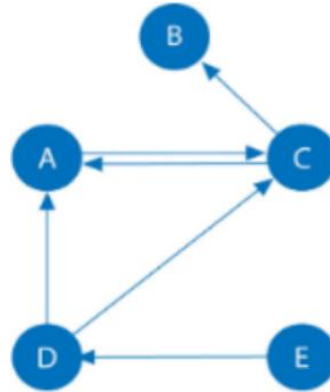


Figura 2.1.2 Gráfica de grafo dirigido

- **Grafo ponderado**

Aquí, cada arista tiene un valor asociado, que llamamos «peso». Este peso puede representar distancia, costo, tiempo, o cualquier otra medida relevante. Es como un mapa real, donde cada calle tiene una longitud específica. [3]

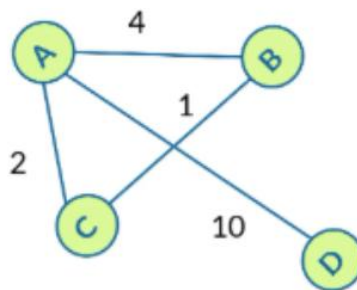


Figura 2.1.3 Gráfica de grafo ponderado

- **Grafo no ponderado**

En estos grafos, todas las conexiones tienen el mismo valor o importancia. Es como si todas las calles en un mapa tuvieran la misma longitud. [3]

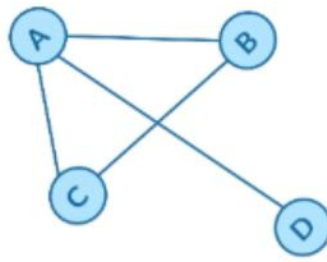


Figura 2.1.4 Gráfica de grafo no ponderado

En los videojuegos su aplicación usualmente se basa en la búsqueda de caminos. La programación de los videojuegos, incluyendo los algoritmos de búsqueda y optimización, la teoría de grafos para la navegación de personajes, el álgebra lineal aplicada a la representación de movimientos y físicas, la estadística y probabilidad en la toma de decisiones, y el aprendizaje autónomo para la mejora continua del comportamiento de los NPCs. Además, se presentan ejemplos prácticos de cómo estas herramientas han permitido el desarrollo de juegos más desafiantes, realistas y personalizados para los jugadores [4].

Aplicación en Videojuegos:

En los videojuegos su aplicación usualmente se basa en la búsqueda de caminos. La programación de los videojuegos, incluyendo los algoritmos de búsqueda y optimización, la teoría de grafos para la navegación de personajes, el álgebra lineal aplicada a la representación de movimientos y físicas, la estadística y probabilidad en la toma de decisiones, y el aprendizaje autónomo para la mejora continua del comportamiento de los NPCs. Además, se presentan ejemplos prácticos de cómo estas herramientas han permitido el desarrollo de juegos más desafiantes, realistas y personalizados para los jugadores [4].

2.2 teoría de arboles

Definición y Propiedades:

En informática, un árbol es una estructura de datos jerárquica formada por nodos, donde cada nodo puede tener cero o más nodos hijos, y exactamente un nodo padre, excepto el nodo raíz, que no tiene padre. Los árboles se utilizan ampliamente para organizar y representar datos que forman jerarquías de forma natural, como sistemas de archivos, organigramas y árboles de sintaxis abstracta. A diferencia de los grafos, los árboles no contienen ciclos; es decir, no es posible regresar al mismo nodo siguiendo un conjunto de aristas. El nodo superior se denomina raíz, y los nodos sin hijos se

llaman hojas. Los árboles son ampliamente utilizados en informática y videojuegos para organizar datos de manera eficiente, representar decisiones, jerarquías y relaciones de dependencia [5].

Árboles Binarios y N-arios:

Los árboles binarios son un tipo de árbol en el que cada nodo puede tener como máximo dos hijos, denominados hijo izquierdo e hijo derecho. Esta estructura es ideal para algoritmos de búsqueda y clasificación, como los árboles de búsqueda binaria (BST), que permiten localizar elementos de forma rápida y eficiente. Por otro lado, los árboles N-arios permiten que cada nodo tenga un número arbitrario de hijos, lo que los hace útiles para representar estructuras más complejas, como menús, mapas de juego o jerarquías de personajes [6].

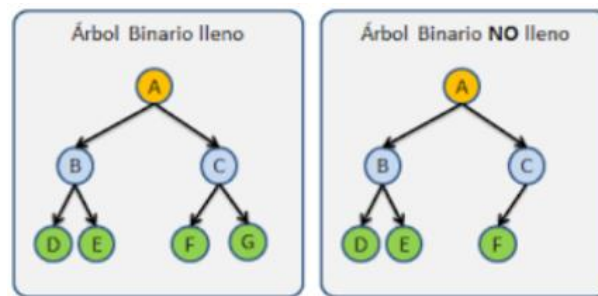


Figura 2.2.1 Árbol n-ario

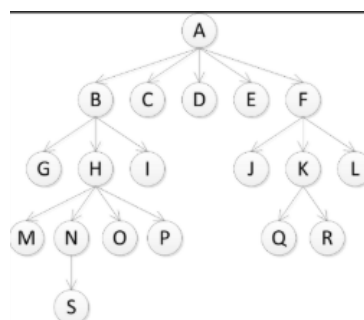


Figura 2.2.2 Árbol binario

Árboles de Decisión:

Los árboles de decisión se utilizan para visualizar y representar procesos de toma de decisiones, como el del ejemplo anterior. En casos de uso más comunes, las técnicas de aprendizaje automático aprovechan de manera similar estos algoritmos de árbol de decisión. Las aplicaciones de aprendizaje automático de árboles de decisión serán el tema central de este artículo [7].



Figura 2.2.3 Árbol de decisiones

2.3 Formatos de Intercambio de Datos (JSON)

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, legible tanto por humanos como por máquinas. Su diseño permite representar información estructurada de manera sencilla y flexible, facilitando la comunicación entre diferentes sistemas, aplicaciones o módulos de un programa [8].

JSON está constituido por pares clave-valor y estructuras de datos básicas como objetos (delimitados por llaves {}) y arreglos o listas (delimitados por corchetes []). Los valores pueden ser de tipo número, cadena de texto, booleano, nulo, objeto o arreglo, lo que permite representar jerarquías complejas de información de manera clara y organizada.

JSON es un formato ligero y legible que permite almacenar y transmitir información entre sistemas o módulos de manera eficiente, independientemente del lenguaje de programación. En videojuegos se utiliza para guardar configuraciones, estados del juego, inventarios y mapas, facilitando la integración con motores como Unity y lenguajes como Python, Java o C# [9].

2.4 Ludología y Diseño de Juegos

Ludología es la disciplina que estudia los juegos y los videojuegos como sistemas interactivos, analizando sus componentes, reglas, mecánicas y efectos en la experiencia del jugador. También implica el diseño de juegos que es el proceso creativo y técnico [10]

En nuestro proyecto, el equipo decidió colectivamente cómo se implementaría el juego desde su planificación inicial, basándonos primero en una temática inspirada en calabozos y dragones para definir las mecánicas principales y funcionamiento general. Cada integrante aportó ideas sobre los tipos de escenarios, enemigos

y las opciones disponibles para el jugador, lo cual permitió estructurar de manera organizada las decisiones de diseño antes de iniciar el desarrollo.

3 ENFOQUE METODOLÓGICO Y PLANIFICACIÓN

3.1 Enfoque del proyecto

Para el desarrollo de nuestro juego "Dungeon of Decisions", adoptamos un enfoque centrado en la implementación práctica de estructuras de datos no lineales (grafos y árboles) dentro de un entorno lúdico. Utilizamos una metodología de desarrollo iterativa e incremental (similar a Agile), lo que nos permitió construir primero el núcleo lógico en python y luego refinar la capa visual paso a paso en Unity. Nuestro juego se ha pensado de tal manera que tiene una arquitectura modular desacoplada: es decir, decidimos separar completamente la Lógica del juego (Backend), que fue programada en Python, de la Capa de Presentación o de interfaz gráfica (Frontend), desarrollada en Unity para una experiencia de usuario mejorada. Esta división nos obligó a diseñar un protocolo de comunicación asíncrono eficiente, simulando un entorno cliente-servidor local mediante intercambio de archivos serializados (es decir lo JSONS).

3.2 Roles del Equipo de Trabajo

Edison:

Como encargado del Backend, asumí la responsabilidad de construir la lógica del juego en Python. Mis tareas específicas incluyeron el diseño y programación de las clases fundamentales (GrafoDungeon, NodoDecision, Jugador), la implementación de los algoritmos de generación procedural para la ruta del mapa (`_generar_ruta_aleatoria(self)`) y la lógica matemática de los combates y eventos. Además, fui responsable de crear el módulo DataManager para gestionar la serialización y deserialización de los archivos JSON, asegurando que los datos fluyeran correctamente entre ambos entornos sin pérdida de información.

Alex:

Alex se encargó del desarrollo visual y la experiencia de usuario en Unity. Su rol consistió en la configuración de la escena gráfica, el diseño de la interfaz de usuario (Canvas, Paneles, Distribución de elementos), y la importación de assets gráficos (sprites y fuentes). Fue el responsable de programar el script GameManager.cs en C#, el cual traduce los datos crudos recibidos del Backend en elementos visuales atractivos (instanciación de

botones, cambio de colores en textos de estado y gestión de sprites dinámicos), garantizando que el jugador recibiera retroalimentación visual inmediata.

Alexander:

Alexander por su parte se encargó de la parte de creación de diálogos en las distintas escenas y demás componentes de la parte gráfica en unity, además dio soporte a Edison en la parte del código en lo que refiere a la creación del código de grafo, árbol y el `gamelogic.py`. Se encargó de investigar y documentar la base matemática de las estructuras de datos utilizadas (Teoría de Grafos y Árboles), justificando su aplicación en el proyecto. A su vez, redactó las descripciones de los eventos, los diálogos de los enemigos y las consecuencias de las decisiones, asegurando que la lógica programada tuviera coherencia argumental y se alineara con los objetivos académicos del informe.

3.3 Herramientas de Desarrollo

A continuación se muestran las tecnologías usadas para el desarrollo del presente proyecto:

Unity (Frontend):

Elegimos Unity (versión 2022/6) como motor gráfico debido a su potente y flexible sistema de UI (Interfaz de Usuario). A diferencia de una consola de texto tradicional, Unity nos permitió implementar elementos visuales dinámicos como botones interactivos, barras de vida con cambio de color y renderizado de sprites, facilitando una experiencia de usuario mucho más amigable y profesional. Su capacidad para ejecutar scripts en C# nos permitió leer archivos externos con facilidad.

Python (Backend):

Seleccionamos Python para la lógica del servidor debido a su sintaxis clara y su excelencia en el manejo de estructuras de datos complejas. Python nos permitió prototipar y depurar rápidamente los algoritmos de los grafos y árboles de decisión sin la sobrecarga de compilación de otros lenguajes como Java. Es ideal para gestionar la lógica matemática (probabilidades, estadísticas, máquinas de estados) que rige las reglas del juego.

JSON:

Utilizamos archivos JSON (`game_state.json` y `player_action.json`) como puente de comunicación. Justificamos su uso por ser un formato ligero, legible por humanos y estándar en la industria para el intercambio de datos. Al ser independiente del lenguaje,

JSON nos permitió serializar objetos complejos en Python y deserializarlos en C# (Unity) sin problemas de compatibilidad, actuando como el enlace vital entre el Backend y el Frontend.

3.4 Planificación Inicial y Diseño

Lluvia de Ideas:

En la fase inicial, descartamos varias ideas para centrar el alcance del proyecto. Originalmente, consideramos hacer un juego de laberinto en 3D con movimiento libre, pero lo descartamos para priorizar la lógica de toma de decisiones basada en grafos sobre la física de movimiento. También descartamos la idea de un multijugador en tiempo real vía red por la complejidad de servidores, optando por un sistema competitivo por turnos ("Hot-seat") donde un jugador completa su ronda y luego cede el turno al siguiente, lo cual simplificó la arquitectura de datos.

Esbozo Final:

Finalmente, elaboramos el documento de diseño definiendo un juego de tipo Roguelite Textual. Establecimos una estructura de 10 Salas (Nodos) lineales generadas con contenido aleatorio. Definimos las mecánicas principales:

- **Estadísticas:** Vida (HP), Oro (Puntaje) y la estadística oculta "Sabiduría/Estrategia".
- **Flujo:** El jugador navega por nodos, enfrenta árboles de decisión (Combate/Evento) y su objetivo es llegar a la Sala 9 (jefe Final) con la mayor cantidad de recursos.
- **Condición de Victoria:** Al finalizar ambos turnos, el sistema compara Sabiduría, Oro y Vida en ese orden jerárquico para declarar al campeón.

A continuación se presenta la propuesta final del juego con las mecánicas que tenemos que implementar y demás aspectos:

DUNGEON OF DECISIONS

1. FICHA TÉCNICA

- **Nombre:** Dungeon of Decisions.
- **Género:** Nuestro juego es RPG (significa Role-Playing game) de Estrategia por Turnos / Aventura de Texto / Roguelite Competitivo (1 vs 1).
- **Arquitectura:**

- **Backend (Cerebro):** Python (Maneja probabilidades, estados, grafos y lógica).
- **Frontend (Visual):** Unity (Maneja la interfaz, animaciones y entrada del usuario).
- **Comunicación:** JSON asíncrono (game_state.json y player_action.json).

2. ESTRUCTURA DEL MUNDO (GRAFOS, ARBOLES Y SALAS)

El juego se genera aleatoriamente en cada partida, pero mantiene una estructura lineal fija de 10 Salas (Nodos).

Tabla de Escenarios y Distribución

La ruta siempre va del Nodo 0 al Nodo 9.

Tabla 3.4.1 Tabla de escenarios (nodos) y distribución de fondos visuales

ID Sala	Tipo de Sala	Contenido Posible (Aleatorio)	Fondo Visual (ID)
0	INICIO	Entrada del Dungeon (Narrativa)	0 (Pasillo Oscuro)
1 - 8	ALEATORIO	Combate (50%) o Evento (50%)	Variable (ver abajo)
9	JEFE FINAL	El Gran Dragón Rojo	9 (Guarida Tesoro)

Nota: La generación aleatoria (_generar_ruta_aleatoria) mezcla una bolsa de encuentros para asegurar variedad en las salas intermedias.

3. ENEMIGOS QUE ENFRENTAREMOS (ENEMIGOS Y DAÑO)

Todos los enemigos tienen estadísticas fijas definidas en DATOS_ENEMIGOS.

Tabla 3.4.2 Tabla de enemigos que enfrentamos en el juego

ID	Nombre	Vida (HP)	Daño Mínimo	Daño Máximo	Costo Soborno
1	Goblin Recluta	20	5	10	30 Oro
2	Orco Berserker	50	10	20	60 Oro
3	Espectro de Hielo	80	15	30	90 Oro

4	Caballero Corrupto	120	20	35	120 Oro
5	GRAN DRAGON ROJO /JEFE FINAL	500	40	60	9999 (Imposible)

4. LÓGICA DE DECISIONES Y CONSECUENCIAS

Cada botón tiene consecuencias matemáticas sobre las 3 estadísticas principales: Vida (HP), Oro (Puntaje) y Sabiduría (Estrategia).

A. Mecánicas de Combate (Enemigos Normales)

Cuando aparece un enemigo (Goblin, Orco, Espectro, Caballero), el jugador tiene 3 opciones:

1. ATACAR (Pelear):

- **Lógica:** Ganas el combate pero recibes daño.
- **Vida:** Resta un valor aleatorio entre el *Daño Mín* y *Daño Máx* del enemigo.
- **Oro:** Ganas una cantidad igual a la HP del enemigo.
- **Sabiduría:** +10 Puntos (Por valentía - es decir enfrentar al enemigo).

2. HUIR (Escapar):

- **Lógica:** Escapas pero te golpean por la espalda.
- **Vida:** -35 HP (Fijo).
- **Oro:** 0.
- **Sabiduría:** -5 Puntos (Por cobardía cuando le da al botón huir).

3. SOBORNAR (Negociar):

- **Costo:** Debes tener el oro indicado en la tabla de enemigos.
- **Probabilidad:**
 - **70% Éxito:** Evitas daño. +30 Sabiduría.
 - **30% Fracaso (Traición):** Pierdes el oro, recibes el daño del ataque y -10 Sabiduría (Ingenuidad).

B. Eventos Especiales (Decisiones)

Cofre Sorpresa

- **Abrir con cuidado (50% Probabilidad):**
 - *Éxito:* +80 Oro, +15 Sabiduría.
 - *Trampa:* -20 HP, 0 Sabiduría.
- **Ignorar:** Avanzas seguro. **+5 Sabiduría** (de qué forma toma el jugador las decisiones).

Altar Místico

- **Ofrecer Plegaria:** +20 HP, +10 Sabiduría (buenas decisiones es más puntaje).
- **Profanar (Robar):** -20 HP (Maldición), +100 Oro, +5 Sabiduría.

Trampa de Suelo

- **Saltar:** +10 Sabiduría.
- **Correr rápido:** -15 HP (Cortes), -5 Sabiduría (Torpeza).

Hada

- **Extender la mano:** +2 Pociones, +10 Sabiduría.
- **Ahuyentar:** Nada ocurre.

El Mercader (Tienda de Riesgo)

- **Comprar Poción:** Costo 50 Oro. +1 Poción, +10 Sabiduría.
- **ROBAR (50% Riesgo):**
 - *Éxito:* +100 Oro, +20 Sabiduría.
 - *Fracaso:* -30 HP (Te rompe un dedo), -15 Sabiduría.
- **Caja Misteriosa:** Costo 30 Oro.
 - *Buena:* +150 Oro, +20 Sabiduría.
 - *Mala (Bomba):* -15 HP, -5 Sabiduría.
- **Vender Vitalidad:** Ganas +50 Oro a cambio de -25 HP y -10 Sabiduría (Desesperación).

Portal del Caos

- **Cruzar el umbral (50% Riesgo):**
 - *Bueno:* Curación Total (Vida = 100), +5 Sabiduría.
 - *Malo:* -40 HP, +5 Sabiduría.
- **Mirar al Vacío:**
 - *Poder:* +15 Sabiduría.
 - *Locura:* -20 HP, -50 Oro, -10 Sabiduría.

Fuente

- **Beber:** +50 HP, +1 Poción, +10 Sabiduría.

5. JEFE FINAL (DRAGÓN ROJO)

En la Sala 9, las reglas cambian. No puedes sobornar.

1. **CANALIZAR ENERGÍA (Misterio):** Fusión con el dragón.
 - **Resultado:** Victoria Suprema. +5000 Oro, +100 Sabiduría, HP restaurada.
2. **DERRUMBAR TECHO (Estrategia):**
 - **Resultado:** Sobrevives. -10 HP, +5000 Oro, +50 Sabiduría.
3. **INTENTAR HUIR:**
 - **Resultado:** MUERTE INSTANTÁNEA (-999 HP). -100 Sabiduría. Game Over.

6. SISTEMA DE OBJETOS Y CASTIGOS

Pociones

- **Efecto:** Al usarla, recuperas +15 HP.
- **Condición:** Debes tener pociones > 0.
- **Límite:** La vida nunca supera 100.

Mecánica de "Necedad " (Main.py)

Si intentas comprar algo (Soborno, Poción, Caja) y no tienes suficiente Oro:

1. **Primer intento:** El sistema te avisa "ORO INSUFICIENTE".

2. **Segundo intento (Insistir):** El sistema te castiga por necio.

- **Consecuencia: -10 HP** (El enemigo aprovecha tu distracción).
- Si mueres por esto, el mensaje es "HAS MUERTO POR TU CODICIA Y OBSTINACIÓN".

7. LÓGICA DEL TORNEO (GANADOR)

El juego es competitivo (Jugador 1 juega primero, luego Jugador 2). Al final, el sistema compara sus estadísticas en este **orden de prioridad estricto**:

1. PRIORIDAD 1: SABIDURÍA (Estrategia)

- Gana quien tenga más puntos de Sabiduría.
- Detalle Gráfico: Si la Sabiduría es negativa (< 0), en la pantalla final aparece el texto "**¡CEREBRO DE GOBLIN!**" en lugar del número.

2. PRIORIDAD 2: ORO (Riqueza)

- Si hay empate en Sabiduría, gana quien tenga más Oro.

3. PRIORIDAD 3: VIDA (Salud)

- Si empatan en todo lo anterior, gana quien tenga más HP restante.

8. RESUMEN DE ESTADOS DE JUEGO (JSON)

Unity recibe estos datos del Backend para dibujar la pantalla:

- vida: Entero (0-100).
- puntaje: Entero (Oro).
- pociones: Entero.
- estrategia: Entero (Puede ser negativo).
- nodo_actual: Entero (0-9).
- imagen_evento_id: Controla qué sprite se muestra (1-10).
- fondo_id: Controla el escenario de fondo (0-12).

- `opciones_disponibles`: Lista de textos para los botones.

4 BACKEND DEL JUEGO (LÓGICA Y ESTRUCTURAS EMPLEADAS)

En este capítulo, detallamos el proceso que planteamos detrás de la parte lógica que aportaría Python al juego (cabe mencionar que todo fue desarrollado en Python). Nuestro objetivo principal fue tratar de entender las reglas matemáticas y narrativas de la interfaz gráfica que habíamos pensado y llevarlas al código lógico en Python, creando un sistema robusto capaz de gestionar estados complejos, fue necesario también recurrir a fuentes externas de consulta como IA, You Tube y libros de programación de juegos que se citan al final de este trabajo. Es así, que diseñamos una arquitectura modular donde la lógica de navegación (Grafos) y la lógica de decisiones (Árboles) operan de manera independiente pero coordinada, asegurando que cada partida sea coherente y libre de errores de ejecución antes de enviar cualquier dato a la pantalla del usuario.

4.1 Implementación del Mapa (Grafos)

Tipo de Grafo Utilizado para el juego:

Para la representación espacial de nuestro juego de Dungeon of Decisions, decidimos implementar un Grafo No Dirigido (Bidireccional). Esta decisión responde a una lógica fundamental de la exploración física de nuestro juego: si un jugador se desplaza de la Sala A a la Sala B, el camino debe permanecer abierto para regresar de B a A, a menos que un evento narrativo específico bloquee el retorno. Además, definimos el grafo como No Ponderado, lo que significa que las aristas que conectan los nodos no tienen un valor de costo o distancia numérica asociado. Priorizamos esto porque el desafío del juego radica en el contenido de cada nodo (enemigos, trampas o tesoros) y no en la distancia recorrida, simplificando así la lógica de navegación y centrando la experiencia en la toma de decisiones inmediata dentro de cada habitación.

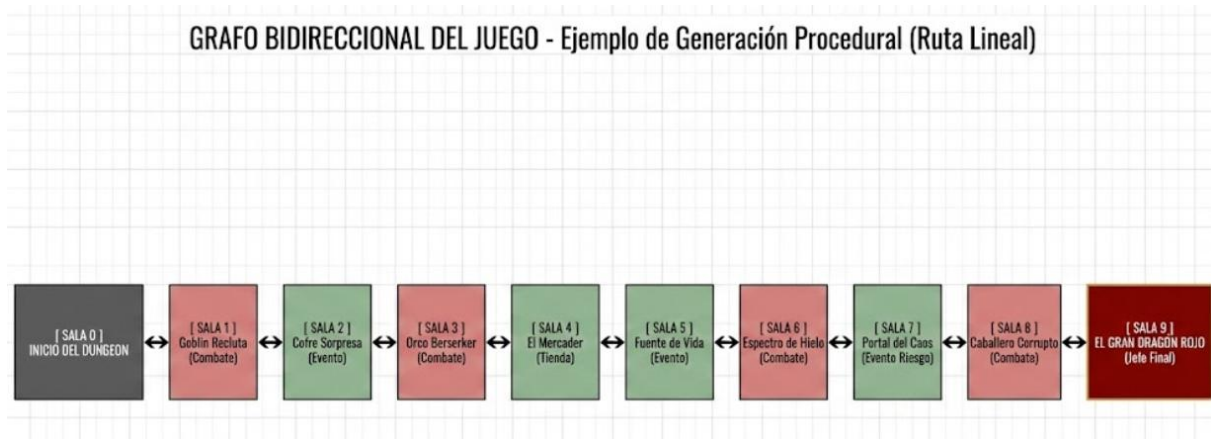


Figura 4.1.1 Vista gráfica del grafo implementado

Código del Grafo:

Para llevar esta teoría al código, creamos la clase GrafoDungeon dentro del archivo graph_system.py. La lógica se centra en un diccionario de nodos donde cada conexión es explícitamente bidireccional. El método crear_conexion es crítico: ya que verifica primero que ambas salas existan en el sistema y luego establece el vínculo mutuo. Esto garantiza que el mapa sea siempre navegable y evita callejones sin salida provocados por lógica unidireccional, proporcionando una base más sólida para los algoritmos de generación procedural que se ejecutan posteriormente al iniciar la partida.

```

# 2. CLASE SALA (Definicion de cada Nodo/ osea de cada sala)
class Sala:
    def __init__(self, id_sala, tipo, dificultad=1):
        self.id = id_sala          # Identificador único del nodo o sala
        self.tipo = tipo           # Tipo de nodo (Enemigo, Tesoro, etc.)
        self.dificultad = dificultad
        self.conexiones = []

        self.arbol_decision = None

    def conectar(self, id_otra_sala):
        if id_otra_sala not in self.conexiones:
            self.conexiones.append(id_otra_sala)

    def asignar_arbol(self, raiz_arbol):
        self.arbol_decision = raiz_arbol

    def __repr__(self):
        return f"Sala[{self.id}] ({self.tipo.value}) -> Conecta con: {self.conexiones}"
  
```

Figura 4.1.2 Vista del código de la clase Sala

```

class GrafoDungeon:
    def __init__(self):
        self.nodos = {}

    def agregar_sala(self, sala):
        self.nodos[sala.id] = sala

    def crear_conexion(self, id_a, id_b, bidireccional=True):
        if id_a in self.nodos and id_b in self.nodos:
            self.nodos[id_a].conectar(id_b)
            if bidireccional:
                self.nodos[id_b].conectar(id_a)
        else:
            print(f"Error: No se pueden conectar {id_a} y {id_b} (¿Seguro que Existen?/Verifíquelo)")

    def obtener_sala(self, id_sala):
        return self.nodos.get(id_sala)

```

Figura 4.1.3 Vista del código de la clase de GrafoDungeon

4.2 Implementación de Decisiones (Árboles)

A continuación se detalla el proceso de implementación de los árboles en nuestro juego:

Tipo de Árbol Utilizado:

Con respecto al flujo narrativo del juego y las opciones del jugador, nos alejamos de las estructuras binarias simples para implementar Árboles N-arios (o Árboles de Decisión Generales). Y , mientras que un árbol binario limita al usuario a respuestas de "Sí/No", nuestro diseño de juego requiera interacciones más ricas, como el evento del Mercader que ofrece hasta cinco acciones distintas (Comprar, Robar, Caja Misteriosa, Vender Sangre, Irse). Al permitir que un nodo padre tenga N cantidad de hijos, logramos una experiencia narrativa mucho más orgánica e impredecible, donde una sola situación puede ramificarse en múltiples desenlaces diversos dependiendo de la estrategia del usuario.

Probabilidades y Azar:

Para evitar que el juego se convirtiera en una experiencia predecible y repetitiva, integramos un componente de azar directamente en los nodos del árbol. Utilizando la librería random de Python, específicamente la función random.random(), programamos una lógica donde ciertas ramas del árbol solo son accesibles base a un cálculo de probabilidad. Por ejemplo, la acción de Robar al Mercader en su escena o sala respectiva tiene un 50% de éxito; el sistema lanza un dado virtual y, según el resultado numérico, asigna dinámicamente el siguiente nodo del árbol, haciendo que cada intento sea único y manteniendo al jugador en tensión constante sobre el resultado de sus acciones arriesgadas, y esto se fundamenta en el hecho de poner al jugador a tomar decisiones

rápidas pero oportunas, que le permitan ganar sabiduría que es uno de los componentes de primera prioridad para determinar al ganador.

Código del Árbol:

La implementación técnica recae en la clase `NodoDecision` del archivo `tree_system.py`. A diferencia de una implementación tradicional de nodo binario con punteros fijos a izquierda y derecha como se ha venido viendo en clase, utilizamos una lista dinámica `self.opciones = []` para almacenar los hijos. Esta flexibilidad nos permite añadir cualquier cantidad de consecuencias a un segmento de historia mediante el desempaquetado de los nodos, creando una estructura escalable que puede crecer en complejidad sin necesidad de refactorizar la clase base como tal del árbol.

```
Backend_Python > tree_system.py > NodoDecision > agregar_opciones
1 class NodoDecision:
2     def __init__(self, texto, es_hoja=False, efecto=None):
3         self.texto = texto
4         self.es_hoja = es_hoja
5         self.efecto = efecto if efecto is not None else {}
6         self.opciones = [] # Ahora usamos una lista flexible, no solo izq/der
7
8     def agregar_opciones(self, *nodos):
9         """
10         Permite agregar cualquier cantidad de opciones.
11         Ejemplo: agregar_opciones(opcion1)
12         Ejemplo: agregar_opciones(atacar, huir, pocion)
13         """
14         for n in nodos:
15             self.opciones.append(n)
16
```

Figura 4.2.1 Vista del código de archivo de tree (árbol)

```
def _arbol_sorpresas(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(texto_intro + "\nUn cofre antiguo reposa en el centro.", efecto={'tipo_enemigo': 0, 'fondo_id': id_fondo})
    roll = random.random()
    if roll > 0.5:
        res = NodoDecision("Abrir con cuidado", es_hoja=False, efecto={'puntaje': 80, 'estrategia': 15, 'texto_custom': "¡FORTUNA! (+80 Oro / +15 HP)"})
    else:
        res = NodoDecision("Abrir con cuidado", es_hoja=False, efecto={'vida': -20, 'estrategia': 0, 'texto_custom': "¡TRAMPA! (-20 HP)"})
    res.agregar_opciones(NodoDecision("Seguir camino", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    raiz.agregar_opciones(res, NodoDecision("Ignorar y seguir", es_hoja=True, efecto={'mover_a_nodo': id_siguiente, 'estrategia': 5}))
    return raiz
```

Figura 4.2.2 Vista del código de árboles de eventos según opción de jugador (árbol)

```
def _arbol_mercader(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(f"{texto_intro}\nEl Mercader te mira con ojos inyectados en sangre.\n'Oro... o Vida. Todo tiene un precio aqui'")

    btn_pocion = NodoDecision("Comprar Pocion (50 Oro)", es_hoja=False,
                               efecto={'costo_oro': 50, 'pocion': 1, 'estrategia': 10, 'texto_custom': "Buena compra. (+1 Pocion / +10 Pt"},
    btn_pocion.agregar_opciones(NodoDecision("Despedirse", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    roll_robo = random.random()
    if roll_robo > 0.5:
        txt_robo = "¡WOW JAJA! Le has quitado la bolsa.\n(+100 Oro / +20 Ptos Estrategia)"
        fx_robo = {'puntaje': 100, 'estrategia': 20, 'texto_custom': txt_robo, 'tipo_enemigo': 9}
    else:
        txt_robo = "¡TE ATRAPO! Te rompe un dedo.\n(-30 HP / -15 Ptos Estrategia)"
        fx_robo = {'vida': -30, 'estrategia': -15, 'texto_custom': txt_robo, 'tipo_enemigo': 9}

    nodo_res_robo = NodoDecision(txt_robo, es_hoja=False, efecto=fx_robo)
    nodo_res_robo.agregar_opciones(NodoDecision("Huir indignado", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    btn_robar = NodoDecision("ROBAR AL MERCADER (50% Riesgo)", es_hoja=False, efecto=nodo_res_robo.efecto)
    for op in nodo_res_robo.opciones: btn_robar.agregar_opciones(op)
```

Figura 4.2.3 Vista del código de árboles de eventos según opción de jugador (árbol mercader)

4.3 Mecánicas de Juego y Código

Sistema de Combate:

El sistema de combate fue diseñado no como una mecánica de acción, sino como un puzzle matemático de gestión de recursos. Definimos diccionarios estáticos para cada tipo de enemigo que contienen su vida máxima y un rango de daño. Cuando el jugador elige "Atacar", el backend garantiza la victoria y la recompensa en oro, pero fuerza un intercambio: el jugador *debe* sacrificar una porción de su vida. Este daño se calcula aleatoriamente entre los umbrales mínimo y máximo del enemigo, generando incertidumbre sobre el costo de la victoria y obligando al jugador a evaluar si tiene suficiente salud para sobrevivir al encuentro.

```
class DungeonLogic:
    DATOS_ENEMIGOS = [
        {"id": 1, "nombre": "Goblin Recluta", "hp": 20, "dano_min": 5, "dano_max": 10, "soborno": 30},
        {"id": 2, "nombre": "Orco Berserker", "hp": 50, "dano_min": 10, "dano_max": 20, "soborno": 60},
        {"id": 3, "nombre": "ESPECTRO DE HIELO", "hp": 80, "dano_min": 15, "dano_max": 30, "soborno": 90},
        {"id": 4, "nombre": "CABALLERO CORRUPTO", "hp": 120, "dano_min": 20, "dano_max": 35, "soborno": 120},
        {"id": 5, "nombre": "GRAN DRAGON ROJO", "hp": 500, "dano_min": 40, "dano_max": 60, "soborno": 9999}
    ]
```

Figura 4.3.1 Vista del código de lista de enemigos a vencer con los puntajes respectivos.

```
def _arbol_combate(self, id_enemigo, id_siguiente, id_fondo, desc_ambiente, desc_victoria):
    datos = next(e for e in self.DATOS_ENEMIGOS if e['id'] == id_enemigo)
    raiz = NodoDecision(f"{desc_ambiente}\nUna figura emerge de las sombras: {datos['nombre']}.", efecto={'tipo_enemigo': id_enemigo, 'fondo_id': id_fondo})

    dano = random.randint(datos['dano_min'], datos['dano_max'])
    oro = datos['hp']
    costo_soborno = datos.get('soborno', 999)

    fx_win_normal = {'vida': -dano, 'puntaje': oro, 'estrategia': 10, 'texto_custom': f"El enemigo cae derrotado. Limpia tu arma.\n(+{oro} Oro"}
    fx_huir = {'vida': -35, 'estrategia': -5, 'texto_custom': "Corres sin mirar atras. (-35 HP / -5 Ptos Estrategia)", 'tipo_enemigo': 0}
```

Figura 4.3.2 Vista del código de árbol combate

Sistema de Economía (Oro y Tienda):

El sistema económico actúa como una segunda capa de restricción en el juego. Cada decisión transaccional (comprar pociones, sobornar) activa una validación en el backend para asegurar que el puntaje (Oro) del jugador sea suficiente. Fuimos un paso más allá implementando una mecánica de Necedad: si un jugador intenta realizar una transacción sin fondos, el sistema primero le advierte, pero ante un segundo intento, aplica un castigo directo a su vida. Esta lógica previene el insistir en una acción sin tener los recursos suficientes, por lo cual el jugador debe monitorear sus recursos cuidadosamente antes de intentar interactuar con las opciones de pago.

```
def _arbol_mercader(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(f"{texto_intro}\nEl Mercader te mira con ojos inyectados en sangre.\n'Oro... o Vida. Todo

    btn_pocion = NodoDecision("Comprar Poción (50 Oro)", es_hoja=False,
                               efecto={'costo_oro': 50, 'pocion': 1, 'estrategia': 10, 'texto_custom': "Buena compra
    btn_pocion.agregar_opciones(NodoDecision("Despedirse", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
```

Figura 4.3.3 Vista del código de árbol mercader (botones)

Sistema de Sabiduría (Puntaje):

Finalmente, diseñamos una estadística oculta llamada "Sabiduría" o "Estrategia", que sirve como el criterio definitivo de desempate para la victoria. A diferencia del Oro o la Vida, que son recursos visibles, la Sabiduría rastrea la *calidad* de las decisiones del jugador. Programamos incrementos positivos para acciones valientes o astutas (como sobornar con éxito) y penalizaciones severas para actos de cobardía (huir) o ilegales (ser atrapado robando). Este valor puede descender por debajo de cero, activando el estado humorístico de "Cerebro de Goblin" (que es cuando la sabiduría se hace negativa), lo que añade una capa de meta-juego donde el comportamiento del usuario es juzgado más allá de su simple supervivencia.

```
# PILAS CUANDO SABIDURA ES NEGATIVA HAGASE ESTO
def formatear_sabiduria(puntos):
    if puntos < 0: return "¡CEREBRO DE GOBLIN!" # Mensaje gracioso si es negativo
    return f"{puntos} Ptos"

txt_e1 = formatear_sabiduria(e1)
txt_e2 = formatear_sabiduria(e2)
```

Figura 4.3.4 Vista del código cuando sabiduría es negativa (botones)

```

btn_derrumbe = NodoDecision("DERRUMBAR TECHO (Estrategia)", es_hoja=False, efecto=nodo_resultado_derrumbe.efecto)
for op in nodo_resultado_derrumbe.opciones: btn_derrumbe.agregar_opciones(op)

die = NodoDecision("INTENTAR HUIR", es_hoja=True, efecto={'vida': -999, 'estrategia': -100, 'texto_custom': "Mueres"})

raiz.agregar_opciones(btn_misterio, btn_derrumbe, die)
return raiz

def intentar_moverse(self, id_destino):
    self.jugador.nodo_actual = id_destino
    return True, "..."

```

Figura 4.3.3 Vista del código de forma en cómo jugador se mueve con ID dado (botones)

5 FRONTEND (UNITY- PARTE GRÁFICA)

5.1 Configuración de la Escena

En la ambientación del juego, se configuro de acuerdo con el tipo de evento que iban a tener. Para esto, la construcción del frontend se baso en no ser solo un modelo estático, sino un entorno que se adapta de acuerdo con la información procesada por la lógica en Python. Esto significa que la escena de Unity no contiene información "quemada", sino que actúa como un contenedor que se rellena en tiempo de ejecución.

Para crear el ambiente correspondiente para nuestro juego (Dungeon of Desicions), se estableció un flujo de trabajo para el trato de los recursos gráficos:

- *Importación de Sprites:* Los elementos visuales (fondos de mazmorras, iconos de pociones y monstruos) se configuraron en Unity bajo el modo de textura "Sprite (2D and UI)". Se utilizaron imágenes de tipo .png.
- *Sistema de Fuentes:* Se implementó el paquete TextMeshPro (TMP) y el uso de la fuente tipo MedievalSharp-Regular, que permiten escalar los textos de vida y narrativa sin que pierdan calidad o se vean pixelados de forma no deseada.

La escena principal se estructuró mediante un Canvas (área donde todos los elementos UI deben estar) en modo Screen Space – Overlay, esto para que siempre esté por encima de cualquier elemento. Se utilizo Canvas Scaler en modo Scale With Screen Scene con resolución 1920x1080 para asegurar que la distribución no se rompa en diferentes monitores.

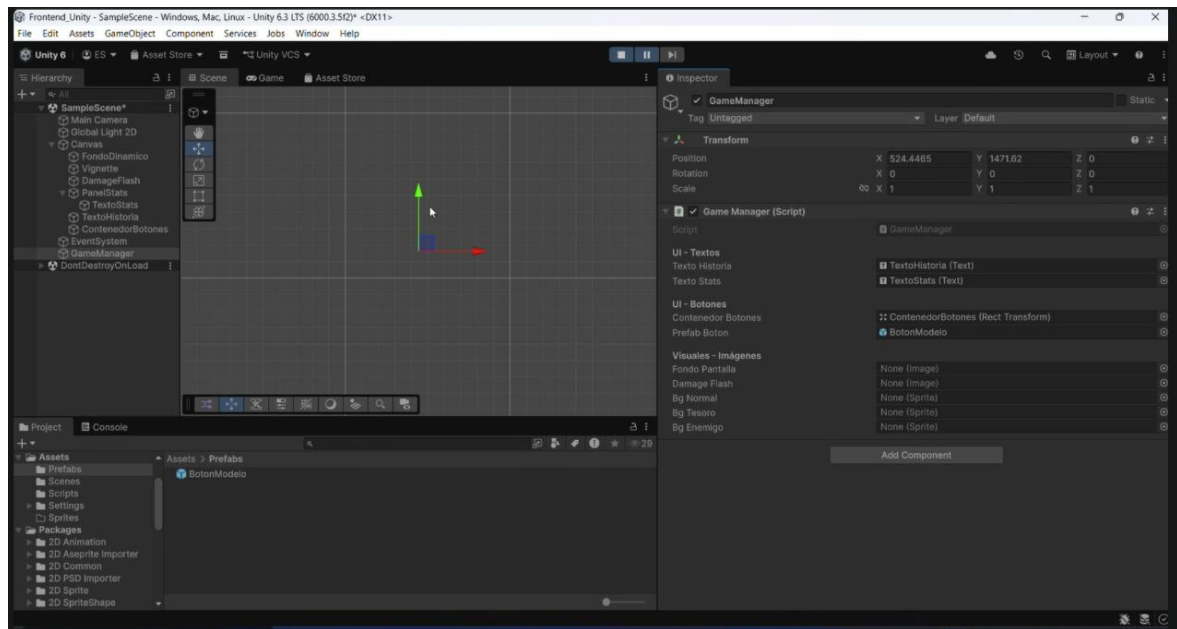


Figura 5.1.1 Vista del panel principal y trazado de escala en Unity

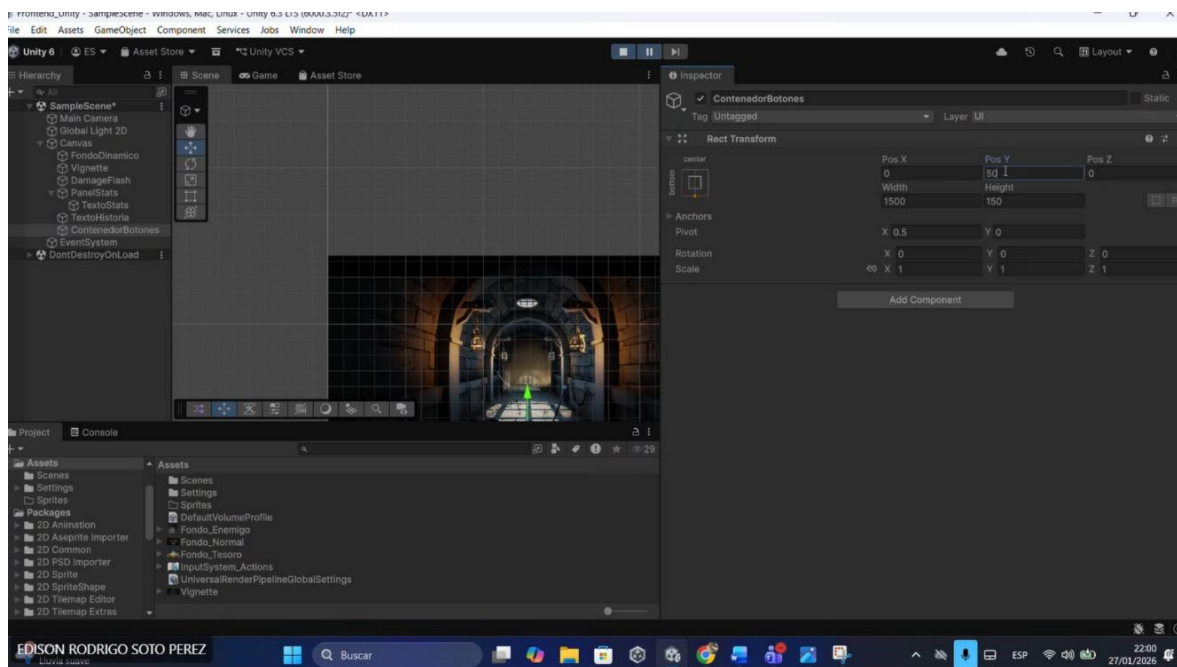


Figura 5.1.2 Vista del panel principal y montaje de escenas en Unity

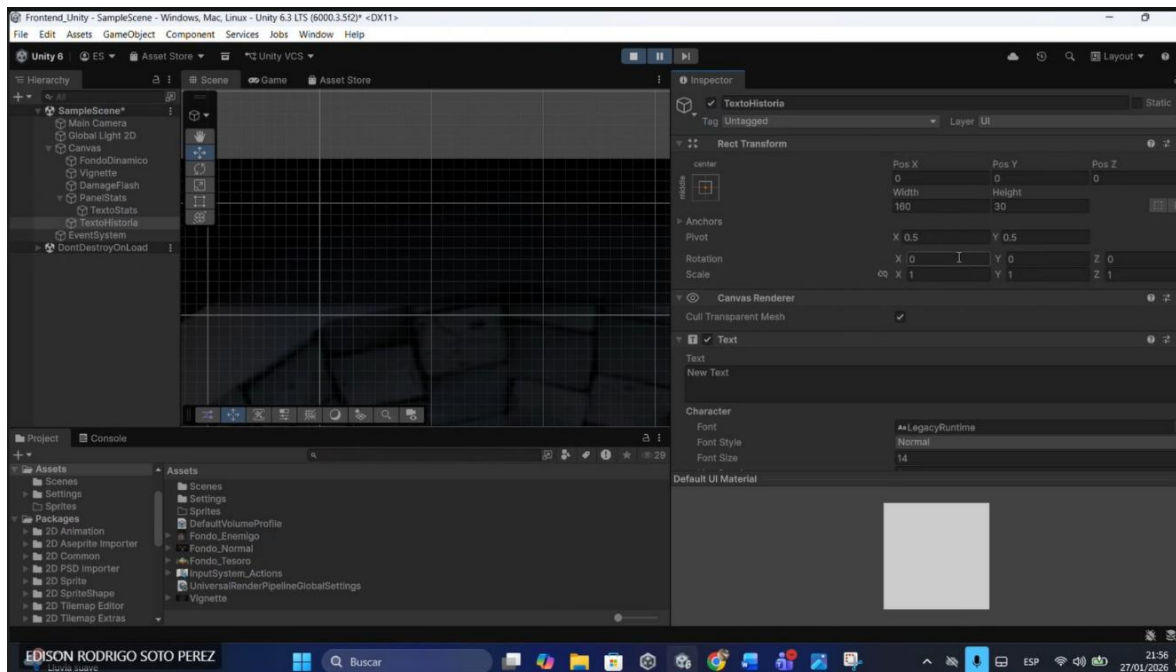


Figura 5.1.3 Vista del panel principal y configuración de escalas

La escena se dividió en tres zonas de responsabilidad:

- *Panel del estado del jugador (Superior)*: Utiliza un Horizontal Layout Group, en donde se muestran los indicadores de vida, dinero recogido, el número de pociones, el número de la sala en la que esta y el botón para usar una poción. Cada indicador tiene un ícono y un texto dinámico que el script GameManager.cs actualiza.

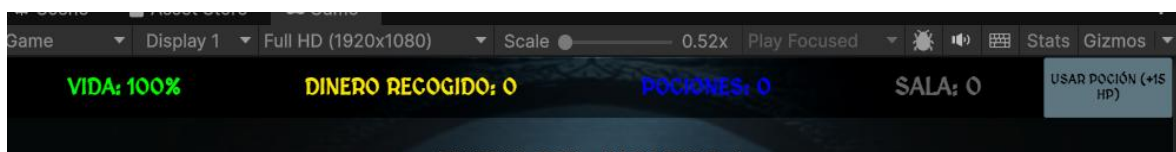


Figura 5.1.4 Vista del panel jugador (Superior)

- *Panel de historia y encuentros (Central)*: Es donde existe mayor peso visual, contiene un objeto Image para mostrar el monstruo o el lugar actual. Además, se muestra una descripción de lo que está ocurriendo en ese momento.

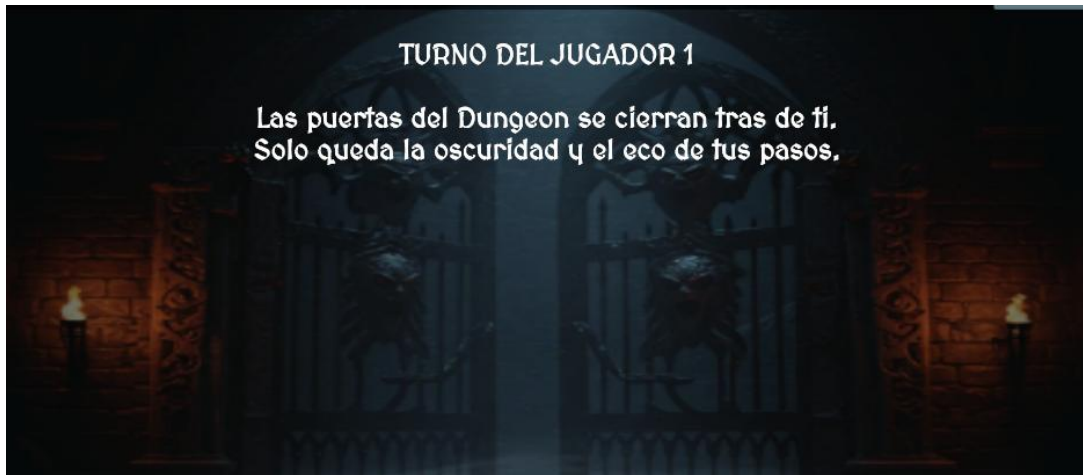


Figura 5.1.5 Vista del panel central (diálogos y personajes, escenarios)

- *Panel de interacción (Inferior):* Este panel está diseñado para la generación dinámica de botones. Se configuró un Horizontal Layout Group con la opción Child Force Expand activada, lo que permite que si hay dos opciones de diálogo, estas se centren automáticamente, y si más de dos se ajusten.

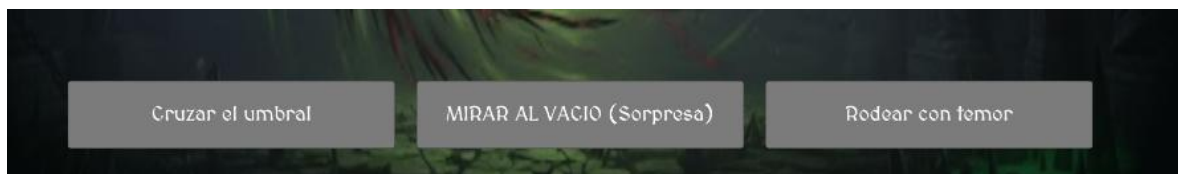


Figura 5.1.6 Vista del panel inferior (botones de desición)

Además, se crearon Prefabs para los botones de opciones. Un Prefab es un objeto "molde" que contiene el diseño, los colores y el script. Esto permite que el GameManager pueda clonar estos botones tantas veces como opciones envíe Python.

5.2 El GameManager

El script GameManager.cs funciona como un controlador maestro. Su propósito no es decidir qué pasa en la historia, sino interpretar el estado del juego que Python escribe en el archivo JSON y traducirlo a elementos visuales de Unity.

Para que el juego sea fluido, el GameManager implementa un ciclo de lectura en un método llamado Update(). Sin embargo, para no saturar el disco duro con miles de lecturas por segundo, se utiliza un temporizador.

El script consulta el archivo game_state.json aproximadamente cada 0.1 a 0.5 segundos. También, se utiliza la librería JsonUtility de Unity para convertir el texto del

JSON en una clase u objeto de C#, permitiendo acceder a datos, jugador, vida como una variable numérica real.

Una de las funcionalidades del GameManager es la gestión del Panel Inferior, en donde se encuentran las decisiones. Dado que Python puede enviar 2, 3 o más opciones según sea la situación, el script realiza lo siguiente:

- *Limpieza*: Borra cualquier botón existente dentro del contenedor de botones.
- *Instanciación*: Por cada opción presente en el JSON, se clona un Prefab del botón preconfigurado.
- *Ingreso de datos*: Asigna el texto de la opción al botón y configura OnClick (Unity) para que envíe el ID de la opción elegido de vuelta a Python a través de player_action.json.

A continuación, se muestra como el script asigna los valores y aplica lógica visual inmediata (como el cambio de color de salud):

```
void ActualizarJuego(GameState estado) {  
    // --- 1. HUD ---  
    if (estado.jugador != null) {  
        textoVida.text = "VIDA: " + estado.jugador.vida + "%";  
        textoOro.text = "DINERO RECOGIDO: " + estado.jugador.puntaje;  
        textoPociones.text = "POCIONES: " + estado.jugador.pociones;  
  
        // --- AQUÍ ESTÁ EL CAMBIO ---  
        // Ahora mostramos la SALA en lugar de la Sabiduría  
        textoSala.text = "SALA: " + estado.jugador.nodo_actual;  
  
        pocionesActuales = estado.jugador.pociones;  
  
        // Color Vida (Verde/Rojo)  
        if (estado.jugador.vida < 30) textoVida.color = Color.red;  
        else textoVida.color = Color.green;  
    }  
}
```

Figura 5.2.1 Fragmento de código donde se asignan los textos y se actualiza el color de vida dependiendo si es baja o no

5.3 Interfaz de Usuario (UI) y UX

Ahora, se explica como la disposición de elementos ayuda a la usabilidad del proyecto.

Horizontal Layout Group: Se utilizó en el Panel de Opciones para que los botones se apilen perfectamente sin necesidad de moverlos manualmente. Si Python envía más texto, el Layout Group ajusta el espaciado automáticamente.

Cuando el jugador presiona un botón, el GameManager envía la acción y bloquea momentáneamente la entrada (Cooldown) para evitar que se procesen múltiples clics por error.

Sistema de colores: Se estableció un código de colores universal:

Amarillo: Para el conteo de oro, resaltando la recompensa.

Rojo/Verde: Para la barra o texto de vida, indicando peligro o salud óptima.

Blanco: Para la narrativa principal, facilitando la lectura prolongada.

6 COMUNICACIÓN E INTEGRACIÓN DEL BACKEND Y EL FRONTEND

En este capítulo explicamos el mecanismo que diseñamos para permitir que dos lenguajes de programación distintos, Python (para el Backend) y C# (Unity para frontend), pudieran compartir la información y la transmisión de datos entre sí de manera fluida. Dado que ambos entornos se ejecutan en procesos separados, optamos por una arquitectura de comunicación asíncrona basada en el intercambio de archivos de texto plano. Para garantizar que los datos fueran legibles y estructurados correctamente, estandarizamos el uso del formato JSON (JavaScript Object Notation). Esta decisión fue importante porque JSON es ligero, universal y permite representar objetos complejos (como el estado de un jugador o una lista de decisiones) de una forma que tanto Python como Unity pueden escribir y leer nativamente sin necesidad de librerías externas pesadas.

6.1 Arquitectura de Archivos JSON

Para mantener el orden en el flujo de datos, establecimos un protocolo unidireccional estricto mediante dos archivos separados: uno exclusivo para la "Salida de Datos" desde el servidor (`game_state.json`) y otro exclusivo para la "Entrada de Datos" del usuario (`player_action.json`). Esta separación evita conflictos de lectura/escritura (condiciones de carrera visto en Sistemas Operativos) y permite depurar el juego fácilmente, ya que podemos abrir estos archivos en cualquier editor de texto y ver exactamente qué está pasando en el cerebro del juego en tiempo real.

`game_state.json`

Este archivo es generado por Python al final de cada ciclo de procesamiento. Su función es actuar como una "fotografía instantánea" de todo lo que ocurre en el juego. Decidimos que Unity no debía realizar ningún cálculo lógico; por lo tanto, este JSON contiene absolutamente toda la información necesaria para el renderizado: desde los puntos de vida y oro actuales, hasta los IDs numéricos que le indican a Unity qué imagen de fondo y qué sprite de enemigo debe dibujar. Además, incluimos la lista dinámica opciones_disponibles, que contiene los textos exactos de los botones que el usuario puede presionar, permitiendo que la interfaz se adapte automáticamente si el árbol de decisión tiene 2, 3 o 5 ramas.

```
game_state.json X
Shared_Data > {} game_state.json > ...
1 {
2   "jugador": {
3     "vida": 47,
4     "puntaje": 50,
5     "nodo_actual": 4,
6     "vivo": true,
7     "pociones": 0
8   },
9   "mensaje_sistema": "...",
10  "texto_evento": "Una tienda improvisada con pieles y huesos.\nEl Mercader te mira",
11  "imagen_evento_id": 9,
12  "fondo_id": 11,
13  "opciones_disponibles": [
14    {
15      "texto": "Comprar Poción (50 Oro)",
16      "accion": "elegir_opcion",
17      "target_id": 0
18    },
19    {
20      "texto": "ROBAR AL MERCADER (50% Riesgo)",
21      "accion": "elegir_opcion",
22      "target_id": 0
23    },
24    {
25      "texto": "CAJA MISTERIOSA (30 Oro)",
26      "accion": "elegir_opcion",
27      "target_id": 0
28    },
29    {
30      "texto": "VENDER VITALIDAD (+50 Oro / -25 HP)",
31      "accion": "elegir_opcion",
32      "target_id": 0
33    },
34    {
35      "texto": "No comprar nada",
36      "accion": "elegir_opcion",
37      "target_id": 0
38    }
39  ]
40 }
```

Figura 6.1.1 Vista de archivo de game_state.json

player_action.json

Por el otro lado, diseñamos este archivo para que sea escrito exclusivamente por Unity cuando el usuario interactúa con la interfaz. Implementamos un sistema de escucha donde

Python monitorea constantemente la existencia de este archivo. Cuando Unity escribe en él, no envía una lógica compleja, sino una "intención" simple. Este JSON: solo contiene el tipo de acción (como "elegir_opcion" o "usar_pocion") y el parámetro necesario (el texto del botón presionado). Una vez que Python detecta y lee este archivo, lo procesa y lo borra o vacía inmediatamente, quedando a la espera de la siguiente interacción del turno, completando así el ciclo de solicitud-respuesta.

```
{  
  "accion": "elegir_opcion",  
  "texto": "Atacar"  
}
```

Figura 6.1.2 Vista de archivo de player_action.json

data_manager.py

Implementamos el script `data_manager.py` para que actúe como el puente de comunicación vital entre nuestro cerebro lógico en Python y la interfaz gráfica en Unity. Importamos la librería `json` porque es el único "idioma" universal que ambos sistemas entienden; gracias a esto, podemos empaquetar todas las estadísticas del jugador, la narrativa y los IDs de las imágenes en un archivo de texto (`game_state.json`) que Unity lee para dibujar la pantalla. Del mismo modo, este script revisa constantemente si existe el archivo `player_action.json` para interpretar qué decisión tomó el usuario en la interfaz, permitiendo un flujo de juego interactivo y fluido mediante la lectura y escritura constante de estos archivos en nuestra carpeta compartida.

```

import json
import os

class DataManager:
    def __init__(self):
        self.ruta_estado = "game_state.json"
        self.ruta_accion = "player_action.json"

    def guardar_estado_juego(self, logic, nodo_actual):
        # Obtenemos opciones
        opciones = []
        for op in nodo_actual.opciones:
            opciones.append({
                "texto": op.texto,
                "accion": "elegir_opcion",
                "target_id": 0
            })

        # Obtenemos el fondo (si no existe, usa 0)
        fondo = getattr(logic, 'fondo_visual_id', 0)

        estado = {
            "jugador": {
                "vida": logic.jugador.vida,
                "puntaje": logic.jugador.puntaje,
                "nodo_actual": logic.jugador.nodo_actual,
                "vivo": logic.jugador.vivo,
                "pociones": logic.jugador.pociones
            },
            "mensaje_sistema": "...",
            "texto_evento": logic.texto_evento,
            "imagen_evento_id": logic.imagen_evento_id,
            "fondo_id": fondo, # ¡CAMPO NUEVO!
            "opciones_disponibles": opciones
        }

```

Figura 6.1.3 Vista de archivo de datamanager.py

7 RESULTADOS Y FUNCIONAMIENTO

7.1 Ejecución Paso a Paso del juego

A continuación se muestra la ejecución del juego de forma ordenada, esto mas que nada con el fin de comprender como se integran los conceptos y códigos anteriores descritos y sobre todo la integración con los archivos JSON respectivos:

Inicio del Juego

Al ejecutar el sistema, Unity muestra un Panel de Tutorial que bloquea la interacción hasta que el usuario comprende las reglas.



Figura 7.1.1 Inicio del videojuego, donde se muestran las reglas

Navegación y Mapa:

A continuación, se muestran algunas escenas de navegación que se pueden encontrar dentro del videojuego. Cada sala, representa un nodo en el grafo:

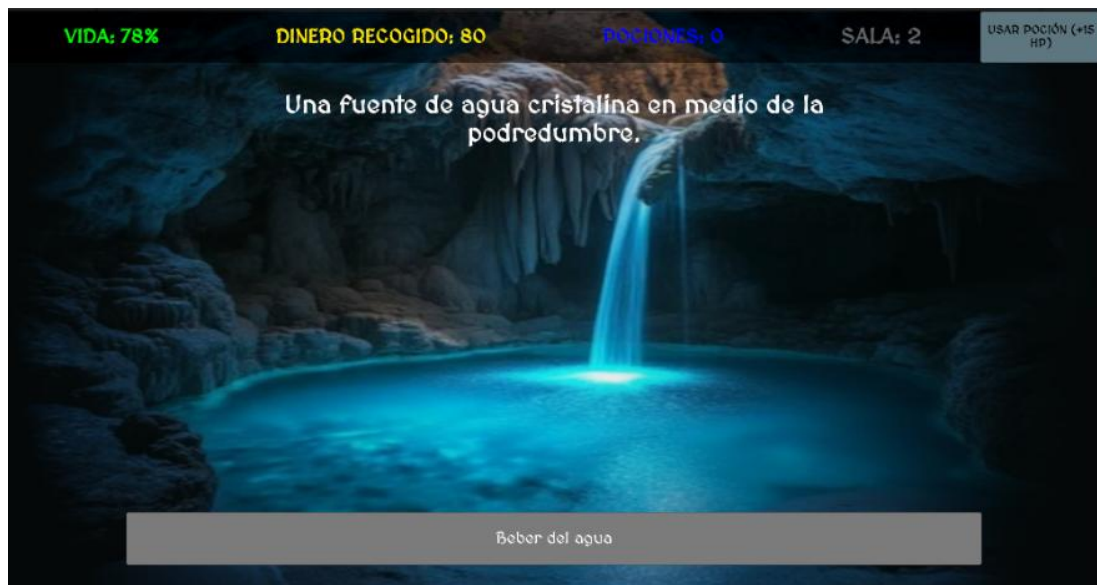


Figura 7.1.2 Fuente

En esta parte, se puede beber el agua y al jugador se le otorgara +50 de salud, 1 poción y +10 de sabiduría.

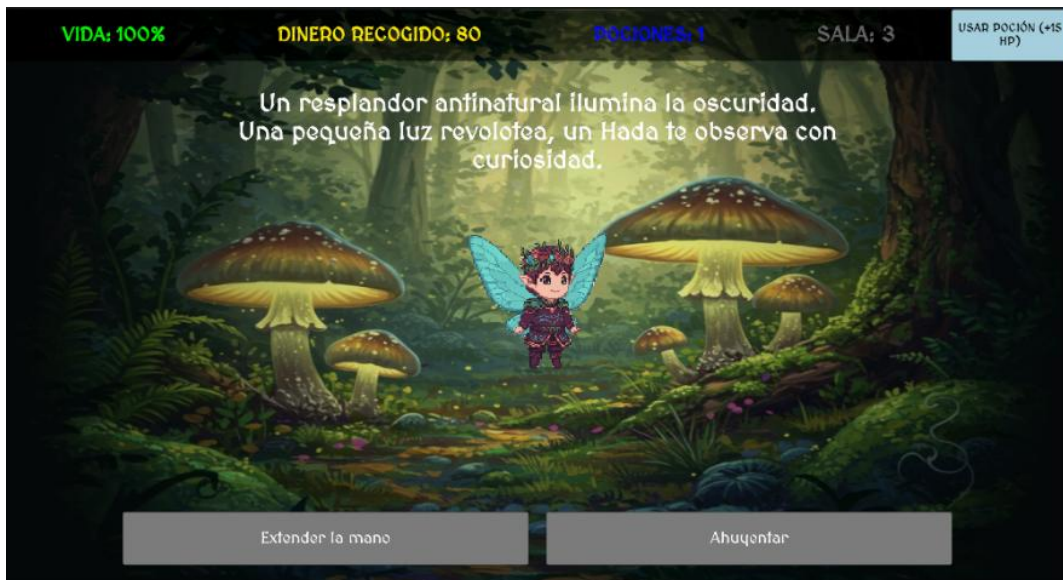


Figura 7.1.3 Escena del Hada

En esta escena de la figura 7.1.3, si extendemos la mano obtendremos 2 pociones y +10 de sabiduría. Así como estas escenas, en el videojuego podremos ir descubriendo algunas más.

Encuentro con Enemigo:



Figura 7.1.4 Enemigo (Espectro de hielo)

Cuando aparece un enemigo (Goblin, Orco, Espectro, Caballero, Dragon), el backend procesa las probabilidades de opciones para cada botón (cabe aclarar que no todos los botones tienen sus árboles de acciones solo algunos botones estratégicos) y devuelve el resultado. El jugador tiene 3 opciones:

- *Atacar:* El jugador gana el combate, pero recibe un daño aleatorio, además, gana la cantidad de oro igual a la salud del enemigo y +10 de sabiduría.
- *Huir (Escapar):* En este caso, el jugador escapa, pero recibe un golpe por la espalda, pierde -35 de salud y -5 de sabiduría.
- *Sobornar (Negociar):* El jugador debe tener el oro indicado en la tabla del enemigo. Existe una probabilidad del 75% que reciba una traición por parte del enemigo, perdiendo el oro, recibiendo daño y -10 de sabiduría. En cambio, con el otro 25%, evita daño y gana +30 de sabiduría.

El sistemas de acciones de los botones y demás aspectos considerados, se recogen en la sección de la planificación que ya fue descrita anteriormente, en ese sentido no se aborda a mayor profundidad este tema en esta sección

Eventos y Tienda

En la sala del Mercader, el sistema de generación de botones de Unity despliega hasta 5 opciones (Comprar poción, robar, no comprar nada, etc.). El sistema verifica el dinero recogido. Si el jugador intenta comprar sin fondos, Python devuelve un mensaje que Unity muestra. En esta escena, se puede comprar una poción con 50 de oro ganando también +10 de sabiduría.

Además, en la caja misteriosa, hay 50/50 de que la caja se buena o mala. Si es buena, el jugador obtiene +150 de oro y +20 de sabiduría, en cambio, si es mala recibe daño reduciendo -15 de salud y -5 de sabiduría. Por último, si se roba al mercader (50% de riesgo), si el robo es exitoso el jugador recibirá +100 de oro y +20 de sabiduría, y si fracasa el jugador pierde -30 de vida y -15 de sabiduría.



Figura 7.1.5 Sala del Mercader

Batalla final:



Figura 7.1.6 Dragón Rojo (jefe Final)

Al igual que con un enemigo normal cuenta con tres opciones (excepto sobornar). Al canalizar energía, obtenemos +5000 de oro, +100 de sabiduría y la salud restaurada. En cambio, si se derrumba el techo, el jugador recibirá +5000 de oro y +50 de sabiduría, pero perderá -10 de salud. Por último, si intenta huir, el jugador morirá instantáneamente, acabando el juego para él.

Victoria o Derrota:

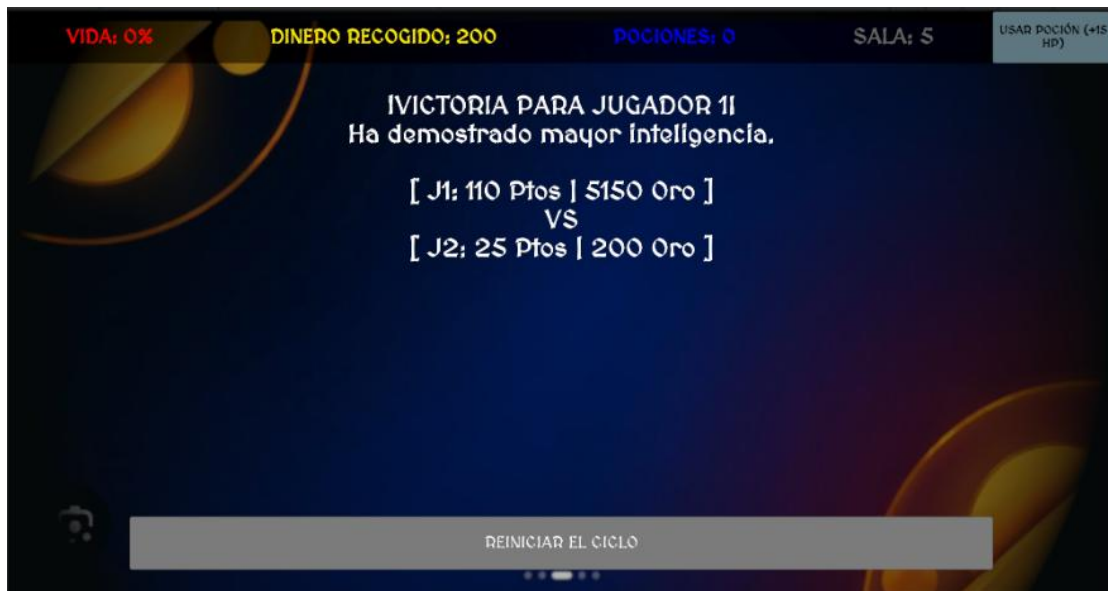


Figura 7.1.7 Resultados finales

Tras concluir el turno del jugador 1 ya sea ganando o perdiendo, el jugador 2 realizara su intento de juego, siendo una competencia entre los dos. Para determinar quién es el ganador, el sistema compara sus estadísticas en este orden de prioridad estricto:

1. **Sabiduría:** Gana quien tenga más puntos de sabiduría.
2. **Oro:** Si existe un empate en Sabiduría, gana quien tenga más oro.
3. **Vida (Salud):** Si empatan en todo lo anterior, gana quien tenga más HP restante.

7.2 Análisis de Resultados

Integración del Flujo de Datos (Backend-Frontend)

Para empezar, el resultado más significativo de nuestro proyecto es la sincronización entre dos entornos de programación completamente distintos. Al ejecutar el sistema, comprobamos que el script `main.py` en Python actúa como un servidor local. Ahora bien, la parte importante ocurre en el intercambio de archivos: observamos que apenas el Backend calcula un evento (por ejemplo, el inicio de un turno como tal), el archivo `game_state.json` se actualiza instantáneamente con la nueva información de vida, oro y narrativa.

En el lado de Unity, el GameManager lee este archivo cada 0.5 segundos. Como resultado, logramos que la interfaz gráfica reaccione casi en tiempo real. Si Python dice que la vida bajó a 20, Unity inmediatamente pinta el texto de color rojo; si Python indica que el enemigo es un Dragón (ID 5), Unity activa el Sprite correspondiente en el lienzo. Esto valida nuestra

ideas previamente investigadas en el marco teórico de que los archivos JSON son un puente eficiente y ligero para este tipo de arquitecturas en juegos.

Validación de la Lógica de Grafos y Árboles

Por otro lado, al realizar las pruebas de juego, confirmamos que la estructura de Grafos No Dirigidos permite una navegación más fluida. El jugador puede avanzar a la Sala 2 y, si la narrativa lo permite, no hay errores técnicos que le impidan regresar conceptualmente, ya que las conexiones bidireccionales en el código `graph_system.py` funcionaron correctamente.

Asimismo, la implementación de Árboles N-arios demostró ser superior a los árboles binarios simples. En las pruebas del evento como la de El Mercader, verificamos que el sistema es capaz de desplegar hasta 5 botones de decisión simultáneos en pantalla. Esto confirma que nuestra clase `NodoDecision` es lo suficientemente flexible para soportar narrativas complejas, permitiendo que el jugador tenga libertad real de elección (Comprar, Robar, Negociar) y no solo caminos lineales de sí/no

Análisis de Mecánicas y Probabilidad

En cuanto a las mecánicas internas, el análisis de las partidas jugadas revela que el factor de azar (implementado con `random` en Python) genera una experiencia de Roguelite auténtica.

Economía y Castigo: Comprobamos que la mecánica de Necesidad funciona como barrera de control. Cuando intentamos comprar una poción sin tener oro suficiente, el sistema primero lanzó la advertencia textual y, al insistir, ejecutó correctamente la resta de vida (-10 HP). Esto demuestra que el Backend tiene control de sí mismo y controla el estado del jugador para evitar trampas o errores de uso.

La Variable de la Sabiduría: Finalmente, el sistema de puntuación funcionó según lo esperado. Al terminar varias partidas de prueba, observamos que ganar no depende solo de la suerte. Jugadores que tomaron decisiones cobardes (como huir constantemente) llegaron al final con vida, pero perdieron la partida debido a su baja puntuación de Sabiduría (Estrategia), validando así que nuestro algoritmo de victoria prioriza la calidad de las decisiones sobre la simple supervivencia.

Estabilidad y Rendimiento

Para finalizar este análisis, sometimos el juego a pruebas de estrés, realizando clics rápidos y transiciones veloces entre salas. El sistema se mantuvo estable; el uso de try-

except en el DataManager evitó que el juego se cerrara si Unity intentaba leer el JSON justo en el milisegundo en que Python lo estaba escribiendo. Aunque existe una latencia imperceptible debido a la lectura del disco, la experiencia de usuario final es fluida y responde adecuadamente a las acciones del jugador.

8 CONCLUSIONES Y RECOMENDACIONES

8.1 Conclusiones

Una de las conclusiones a las que hemos llegado como equipo es que implementar un Grafo No Dirigido fue sumamente importante para transformar una lista abstracta de números en un mundo explorable y coherente. Al concebir las salas como nodos y los pasillos como conexiones bidireccionales, logramos que el jugador sintiera libertad real de movimiento, pudiendo avanzar hacia el peligro o retroceder a una zona segura si la situación lo requería. Sin esta estructura de grafo, el juego habría sido una experiencia lineal y restrictiva (como una simple secuencia de diapositivas); gracias al grafo, creamos un verdadero esquema de juego sencillo donde la navegación espacial es parte fundamental de la estrategia del usuario y de su ganancia en el mismo como tal.

Otra conclusión es que un grafo del tipo no dirigido fue importante para la jugabilidad, ya que resolvió automáticamente el problema de los bloqueos lógicos. Si hubiéramos usado punteros simples o listas enlazadas, el jugador podría haber quedado atrapado en una habitación sin salida, rompiendo la experiencia. La estructura que implementamos garantiza matemáticamente que, si puede entrar a una sala, también puede salir de ella. Esto nos enseñó que elegir el tipo correcto de grafo no es solo una decisión técnica de código, sino una decisión de diseño que asegura que la partida fluya sin errores y sin frustrar al jugador.

Respecto a la toma de decisiones, concluimos que los Árboles N-arios fueron la herramienta que permitió que la historia dejara de ser binaria (simple "Sí o No") para volverse más interesante. Al permitir que cada rama de la historia (Nodo) tuviera múltiples hijos, pudimos diseñar encuentros complejos como el del Mercader, donde el jugador tiene hasta cinco opciones distintas simultáneas. Esto nos demostró que los árboles no sirven solo para ordenar datos, sino que son buenos para modelar conversaciones y consecuencias, permitiendo que una sola situación narrativa explote en múltiples desenlaces diferentes según la elección del usuario y de la probabilidad de ocurrencia de eventos establecida en el código.

Descubrimos que estructurar los eventos en forma de árboles nos dio también una ventaja a la hora de crear el contenido: la escalabilidad. Gracias a la clase `NodoDecision`, agregar una nueva opción a un evento no requería reescribir todo el código, sino simplemente conectar una nueva rama al árbol existente. Esta modularidad nos permitió expandir las mecánicas del juego fácilmente y nos enseñó que una buena estructura de datos separa la lógica de programación del contenido creativo, facilitando el trabajo del equipo de desarrollo a largo plazo.

Una conclusión crucial es cómo logramos que los árboles de decisión se sintieran "vivos" al inyectarles probabilidad. Al combinar la estructura estática del árbol con el azar (funciones `random`), logramos que recorrer la misma "rama" del árbol (por ejemplo, intentar robar) no siempre llevara al mismo nodo de destino (éxito o fracaso). Esto transformó una estructura de datos rígida en una mecánica de juego emocionante y rejugable, demostrando que los algoritmos pueden alterar el recorrido de un árbol en tiempo real para generar incertidumbre y sorpresa en el jugador.

Finalmente, concluimos que nuestro juego radica en la cooperación perfecta entre ambas estructuras: el Grafo que gestiona el contexto macro (dónde estoy y a dónde puedo ir) y el Árbol gestiona el contexto micro (qué está pasando aquí y qué puedo hacer). Entendimos que un RPG (Role-playing game) se construye superponiendo estas capas: el jugador navega por el grafo para encontrar nodos, y dentro de cada nodo del grafo, se despliega un árbol de decisión. Esta arquitectura híbrida es lo que permitió crear una experiencia completa, probando que la combinación de diferentes estructuras de datos es mucho más potente que el uso aislado de cada una.

8.2 Recomendaciones

En primer lugar, sugerimos ampliar la complejidad del diseño del mapa aprovechando mejor la estructura de grafos que ya hemos programado. Aunque el diseño actual de diez salas lineales funcionó perfectamente para validar el prototipo, creemos que una versión futura debería implementar un grafo tipo "telaraña" o laberinto con múltiples rutas alternas y conexiones cruzadas. Esto permitiría al jugador tomar decisiones estratégicas reales sobre su navegación, eligiendo entre rutas cortas pero peligrosas o caminos largos pero más seguros, sacándole así el máximo provecho a la capacidad técnica de los nodos bidireccionales que desarrollamos en el backend.

Por otro lado, recomendamos enriquecer los árboles de decisión integrando un sistema de Clases de Personaje al inicio de la partida, como permitir al jugador elegir entre ser un Guerrero o un Mago. Actualmente, las ramas narrativas son iguales para todos, pero sería

muy interesante programar condiciones específicas en los nodos del árbol donde ciertas acciones sean exclusivas o tengan bonificaciones; por ejemplo, que solo un personaje mágico pueda interactuar con el Altar o que un pícaro tenga éxito garantizado al robar al Mercader. Esto elevaría la rejugabilidad y demostraría la flexibilidad de nuestros árboles N-arios para manejar lógica condicional avanzada.

Finalmente, aconsejamos potenciar el apartado estético en el Frontend agregando una capa de retroalimentación audiovisual, incluyendo efectos de sonido y música ambiental dinámica. A pesar de que logramos que la lógica matemática y la conexión con Python fueran sólidas y estables, notamos que la inmersión del usuario mejoraría drásticamente con detalles sensoriales, como sonidos de impacto al atacar o una música de tensión al entrar a la sala del Dragón. Dado que Unity cuenta con herramientas nativas de audio muy accesibles, esta mejora se podría integrar fácilmente con nuestro GameManager actual sin poner en riesgo la estabilidad del código lógico que ya hemos construido.

9 REFERENCIAS BIBLIOGRÁFICAS

- [1] Erickson. J, "What is JSON?", Oracle, (2024, 4 abril). Disponible en : <https://www.oracle.com/latam/database/what-is-json/>
- [2] Nava. J, "¿Qué es Unity y para qué sirve este motor gráfico?", Empower Talent, (2024). Disponible en: <https://empowertalent.com/que-es-unity/>
- [3] Mozilla Developer Network, "Trabajar con JSON - Aprendiendo desarrollo web," *MDN Web Docs*, 2023. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Learn/JavaScript/Objects/JSON>
- [4] OpenWebinars, "Qué son los grafos y qué tipos existen en programación," *OpenWebinars.net*, 2021. [En línea]. Disponible en: <https://openwebinars.net/blog/que-son-los-grafos-y-tipos-que-existen/>
- [5] KeepCoding, "Árboles en programación: Qué son, tipos y para qué sirven," *KeepCoding Tech School*, 2022. [En línea]. Disponible en: <https://keepcoding.io/blog/arboles-en-programacion/>
- [6] R. Márquez, "Qué es un Roguelike y en qué se diferencia de un Roguelite: la guía definitiva," *VidaExtra*, 2019. [En línea]. Disponible en: <https://www.vidaextra.com/rpg/que-roguelike-que-se-diferencia-roguelite>

- [7] Unity Technologies, "Manual de Unity: Diseño de Interfaces de Usuario (UI Toolkit y Canvas)," *Unity Documentation*, 2022. [En línea]. Disponible en: <https://docs.unity3d.com/es/2022.1/Manual/UI.html>
- [8] Python Software Foundation, "Estructuras de datos y algoritmos en Python," *Documentación oficial de Python 3.12*, 2024. [En línea]. Disponible en: <https://docs.python.org/es/3/tutorial/datastructures.html>
- [9] Microsoft, "Información general sobre la serialización de JSON: Cómo serializar y deserializar," *Microsoft Learn (Documentación .NET)*, 2023. [En línea]. Disponible en: <https://learn.microsoft.com/es-es/dotnet/standard/serialization/system-text-json/overview>
- [10] J. M. Alarcón, "Cómo funcionan los diccionarios (Hash Maps) y por qué son tan eficientes en la búsqueda de datos," *CampusMVP.es*, 2020. [En línea]. Disponible en: <https://www.campusmvp.es/recursos/post/como-funcionan-los-diccionarios-hash-maps.aspx>
- [11] El Libro de Python, "Manejo de Archivos, Lectura y Escritura (Context Managers)," *Ellibrodepython.com*, 2022. [En línea]. Disponible en: <https://ellibrodepython.com/archivos-python>
- [12] Unity Technologies, "Scripting API: MonoBehaviour.StartCoroutine y el manejo del tiempo," *Unity Documentation*, 2023. [En línea]. Disponible en: <https://docs.unity3d.com/es/2021.3/ScriptReference/MonoBehaviour.StartCoroutine.html>
- [13] A. S. Tanenbaum, "Estructuras de datos dinámicas: Listas y Árboles en el desarrollo de software," *Facultad de Informática UCM*, 2019. [En línea]. Disponible en: <https://informatica.ucm.es/estructuras-datos-dinamicas>
- [14] E. GraphEverywhere, "Grafos | Qué son, tipos, orden y herramientas de visualización", GraphEverywhere, 11 de febrero de 2021. Disponible en: <https://www.grapheverywhere.com/grafos-que-son-tipos-orden-y-herramientas-de-visualizacion/>
- [15] J. P. Porto y A. Gardey, "Ludología - Qué es, definición y concepto", *Definición.de*, 23 de mayo de 2022. Disponible en: <https://definicion.de/ludologia/>

10 ANEXOS

ANEXO I. Script de python del árbol de decisiones (tree_system.py)

```
class NodoDecision:
    def __init__(self, texto, es_hoja=False, efecto=None):
        self.texto = texto
        self.es_hoja = es_hoja
        self.efecto = efecto if efecto is not None else {}
        self.opciones = [] # Ahora usamos una lista flexible, no solo izq/der

    def agregar_opciones(self, *nodos):
        """
        Permite agregar cualquier cantidad de opciones.
        Ejemplo: agregar_opciones(opcion1)
        Ejemplo: agregar_opciones(atacar, huir, pocion)
        """
        for n in nodos:
            self.opciones.append(n)

    @property
    def izquierda(self):
        return self.opciones[0] if len(self.opciones) > 0 else None

    @property
    def derecha(self):
        return self.opciones[1] if len(self.opciones) > 1 else None
```

ANEXO II. Script de python del grafo del videojuego (graph_system.py)

```
from enum import Enum

# 1. DEFINICIÓN DE TIPOS (Osea los vértices del Grafo)
class TipoSala(Enum):
    NORMAL = "Normal"
    ENEMIGO = "Enemigo"
    POCION = "Pocion"
    TESORO = "Tesoro"
    EVENTO = "Evento Aleatorio"
    TRAMPA = "Trampa"
    FINAL = "Jefe Final"

# 2. CLASE SALA (El Nodo)
class Sala:
    def __init__(self, id_sala, tipo, dificultad=1):
        self.id = id_sala # Identificador único del nodo
        self.tipo = tipo # Tipo de nodo (Enemigo, Tesoro, etc.)
        self.dificultad = dificultad
        self.conexiones = []

        self.arbol_decision = None

    def conectar(self, id_otra_sala):
        if id_otra_sala not in self.conexiones:
            self.conexiones.append(id_otra_sala)

    def asignar_arbol(self, raiz_arbol):
        self.arbol_decision = raiz_arbol

    def __repr__(self):
        return f"Sala[{self.id}] ({self.tipo.value}) -> Conecta con: {self.conexiones}"
```

```

class GrafoDungeon:
    def __init__(self):
        self.nodos = {}

    def agregar_sala(self, sala):
        self.nodos[sala.id] = sala

    def crear_conexion(self, id_a, id_b, bidireccional=True):
        if id_a in self.nodos and id_b in self.nodos:
            self.nodos[id_a].conectar(id_b)
            if bidireccional:
                self.nodos[id_b].conectar(id_a)
        else:
            print(f"Error: No se pueden conectar {id_a} y {id_b} (¿Existen?)")

    def obtener_sala(self, id_sala):
        return self.nodos.get(id_sala)

```

ANEXO III. Script de python del manejo de datos (data_manager.py)

```

import json
import os

class DataManager:
    def __init__(self):
        self.ruta_estado = "game_state.json"
        self.ruta_accion = "player_action.json"

    def guardar_estado_juego(self, logic, nodo_actual):
        # Obtenemos opciones
        opciones = []
        for op in nodo_actual.opciones:
            opciones.append({
                "texto": op.texto,
                "accion": "elegir_opcion",
                "target_id": 0
            })

        # Obtenemos el fondo (si no existe, usa 0)
        fondo = getattr(logic, 'fondo_visual_id', 0)

        estado = {
            "jugador": {
                "vida": logic.jugador.vida,
                "puntaje": logic.jugador.puntaje,
                "nodo_actual": logic.jugador.nodo_actual,
                "vivo": logic.jugador.vivo,
                "pociones": logic.jugador.pociones
            },
            "opcion": " ",
            "fondo": " "
        }

```

ANEXO IV. Script de lógica de Python (game_logic.py)

```
import random
from graph_system import GrafoDungeon, Sala, TipoSala
from tree_system import NodoDecision

class Jugador:
    def __init__(self, nodo_inicial):
        self.vida = 100
        self.puntaje = 0 # ORO
        self.estrategia = 0 # PUNTOS DE BUENAS DECISIONES
        self.pociones = 0
        self.fuerza = 0
        self.defensa = 0
        self.nodo_actual = nodo_inicial
        self.nodo_anterior = 0
        self.vivo = True

    def modificar_vida(self, cantidad):
        self.vida += cantidad
        if self.vida > 100: self.vida = 100
        if self.vida <= 0:
            self.vida = 0
            self.vivo = False
        else:
            self.vivo = True

class DungeonLogic:
    DATOS_ENEMIGOS = [
        {"id": 1, "nombre": "Goblin Recluta", "hp": 20, "dano_min": 5, "dano_max": 10, "soborno": 30},
        {"id": 2, "nombre": "Orco Berserker", "hp": 50, "dano_min": 10, "dano_max": 20, "soborno": 60},
        {"id": 3, "nombre": "ESPECTRO DE HIELO", "hp": 80, "dano_min": 15, "dano_max": 30, "soborno": 90},
        {"id": 4, "nombre": "CABALLERO CORRUPTO", "hp": 120, "dano_min": 20, "dano_max": 35, "soborno": 120},
        {"id": 5, "nombre": "GRAN DRAGON ROJO", "hp": 500, "dano_min": 40, "dano_max": 60, "soborno": 999}
    ]
]
```

```
def __init__(self):
    self.grafo = GrafoDungeon()
    self.jugador = None
    self.texto_evento = ""
    self.imagen_evento_id = 0
    self.turno_jugador = 1

    # Datos finales
    self.vida_jugador_1 = 0
    self.oro_jugador_1 = 0
    self.est_jugador_1 = 0

    self.vida_jugador_2 = 0
    self.oro_jugador_2 = 0
    self.est_jugador_2 = 0

def iniciar_juego(self):
    self.jugador = Jugador(0)
    self._generar_ruta_aleatoria()
    self.imagen_evento_id = 0
    print(f"--- SISTEMA: Turno Jugador {self.turno_jugador} Iniciado ---")

def reiniciar_torneo(self):
    print("--- REINICIANDO TORNEO COMPLETO ---")
    self.turno_jugador = 1
    self.vida_jugador_1 = 0
    self.oro_jugador_1 = 0
    self.est_jugador_1 = 0
    self.vida_jugador_2 = 0
    self.oro_jugador_2 = 0
    self.est_jugador_2 = 0
    self.iniciar_juego()
    self.texto_evento = "NUEVA PARTIDA: JUGADOR 1, TU TURNO."
```

```

def finalizar_turno_actual(self, causa_muerte=False):
    vida_final = self.jugador.vida if not causa_muerte else 0
    oro_final = self.jugador.puntaje
    est_final = self.jugador.estrategia

    if self.turno_jugador == 1:
        self.vida_jugador_1 = vida_final
        self.oro_jugador_1 = oro_final
        self.est_jugador_1 = est_final

        txt_resumen = f"EL JUGADOR 1 HA CAIDO.\n(HP: {vida_final} | Oro: {oro_final} | Sabiduria: {est_final})"
        if not causa_muerte:
            txt_resumen = f"EL JUGADOR 1 HA SOBREVIVIDO.\n(HP: {vida_final} | Oro: {oro_final} | Sabiduria: {est_final})"

        raiz = NodoDecision(f"{txt_resumen}\n\nAHORA EL DESTINO LLAMA AL JUGADOR 2.\n¿Tienes el valor suficiente?",
                           efecto={'tipo_enemigo': 0, 'fondo_id': 0})
        btn_p2 = NodoDecision("TOMAR EL LUGAR", es_hoja=True,
                              efecto={'cambiar_turno': True, 'texto_custom': "Iniciando J2..."})
        raiz.agregar_opciones(btn_p2)
        return raiz
    else:
        self.vida_jugador_2 = vida_final
        self.oro_jugador_2 = oro_final
        self.est_jugador_2 = est_final
        return self._calcular_ganador()

def _calcular_ganador(self):
    v1 = self.vida_jugador_1
    o1 = self.oro_jugador_1
    e1 = self.est_jugador_1

    v2 = self.vida_jugador_2
    o2 = self.oro_jugador_2
    e2 = self.est_jugador_2

```

```

# --- AQUÍ ESTÁ EL ARREGLO GRACIOSO ---
def formatear_sabiduria(puntos):
    if puntos < 0: return "¡CEREBRO DE GOBLIN!" # Mensaje gracioso si es negativo
    return f"{puntos} Ptos"

txt_e1 = formatear_sabiduria(e1)
txt_e2 = formatear_sabiduria(e2)

detalles = f"\n\n J1: {txt_e1} | {o1} Oro ]\nVS\n J2: {txt_e2} | {o2} Oro ]"

if e1 > e2:
    ganador = f"¡VICTORIA PARA JUGADOR 1!\nHa demostrado mayor inteligencia.{detalles}"
elif e2 > e1:
    ganador = f"¡VICTORIA PARA JUGADOR 2!\nSus decisiones fueron mas sabias.{detalles}"
else:
    if o1 > o2:
        ganador = f"¡EMPATE EN SABIDURIA!\nPero JUGADOR 1 gana por RIQUEZA.{detalles}"
    elif o2 > o1:
        ganador = f"¡EMPATE EN SABIDURIA!\nPero JUGADOR 2 gana por RIQUEZA.{detalles}"
    else:
        if v1 > v2:
            ganador = f"EMPATE TOTAL... Pero J1 esta mas sano. ¡Victoria J1!{detalles}"
        elif v2 > v1:
            ganador = f"EMPATE TOTAL... Pero J2 esta mas sano. ¡Victoria J2!{detalles}"
        else:
            ganador = f"EMPATE ABSOLUTO Y LEGENDARIO.{detalles}"

raiz = NodoDecision(ganador, efecto={'tipo_enemigo': 7, 'fondo_id': 9})
btn_reset = NodoDecision("REINICIAR EL CICLO", es_hoja=True,
    efecto={'reiniciar_total': True, 'texto_custom': "El tiempo retrocede..."})
raiz.agregar_opciones(btn_reset)
return raiz

```

```

def obtener_nodo_game_over(self):
    return self.finalizar_turno_actual(causa_muerte=True)

def _generar_ruta_aleatoria(self):
    self.grafo.salas = {}
    for i in range(10): self.grafo.agregar_sala(Sala(i, TipoSala.NORMAL))
    for i in range(9): self.grafo.crear_conexion(i, i+1)

    self.grafo.obtener_sala(0).asignar_arbol(self._arbol_inicio())
    self.grafo.obtener_sala(9).asignar_arbol(self._arbol_jefe_final(5, 0))

    combates = [
        (self._arbol_combate, 1, 1, "El aire huele a humedad y sangre vieja.", "El Goblin exhala su ultimo aliento."),
        (self._arbol_combate, 2, 3, "El calor es insoportable. Hay huesos en el suelo.", "El Orco cae con un ruido sordo."),
        (self._arbol_combate, 3, 5, "Una niebla helada congela tus pulmones.", "El espectro se disuelve en el aire."),
        (self._arbol_combate, 4, 7, "Estandartes rotos y metal oxidado.", "La armadura vacia se desploma.")
    ]

    eventos = [
        (self._arbol_sorpresa, 2, "Un pasillo largo cubierto de polvo de siglos."),
        (self._arbol_evento_mistico, 4, "Un altar antiguo con inscripciones prohibidas."),
        (self._arbol_evento_trampa, 6, "El suelo tiene patrones extraños e irregulares."),
        (self._arbol_hada, 10, "Un resplandor antinatural ilumina la oscuridad."),
        (self._arbol_mercader, 11, "Una tienda improvisada con pieles y huesos."),
        (self._arbol_portal_caos, 12, "La realidad se distorsiona frente a ti."),
        (self._arbol_fuente, 8, "Una fuente de agua cristalina.")
    ]

    bolsa_total = combates + eventos
    random.shuffle(bolsa_total)
    seleccionados = bolsa_total[:8]

```

```

    for i in range(1, 9):
        data = seleccionados[i-1]
        func = data[0]
        if func == self._arbol_combate:
            arbol = func(data[1], i+1, data[2], data[3], data[4])
        elif func == self._arbol_fuente:
            arbol = func(i+1)
        else:
            arbol = func(i+1, data[1], data[2])
        self.grafo.obtener_sala(i).asignar_arbol(arbol)

# --- ARBOLES DE DECISIÓN ---

def _arbol_inicio(self):
    txt = f"TURNO DEL JUGADOR {self.turno_jugador}\n\nLas puertas del Dungeon se cierran tras de ti.\nSolo queda la oscuridad y"
    raiz = NodoDecision(txt, es_hoja=False, efecto={'fondo_id': 0})
    raiz.agregar_opciones(NodoDecision("Adentrarse en la oscuridad", es_hoja=True, efecto={'mover_a_nodo': 1}))
    return raiz

def _arbol_combate(self, id_enemigo, id_siguiete, id_fondo, desc_ambiente, desc_victoria):
    datos = next(e for e in self.DATOS_ENEMIGOS if e['id'] == id_enemigo)
    raiz = NodoDecision(f"{desc_ambiente}\nUna figura emerge de las sombras: {datos['nombre']}.", efecto={'tipo_enemigo': id_enemigo})

    dano = random.randint(datos['dano_min'], datos['dano_max'])
    oro = datos['hp']
    costo_soborno = datos.get('soborno', 999)

    fx_win_normal = {'vida': -dano, 'puntaje': oro, 'estrategia': 10, 'texto_custom': f"El enemigo cae derrotado. Limpia tu arma"}
    fx_huir = {'vida': -35, 'estrategia': -5, 'texto_custom': "Corres sin mirar atras. (-35 HP / -5 Ptos Estrategia)", 'tipo_enemigo': id_enemigo}

    dado_destino = random.random()
    if dado_destino > 0.7:
        txt_res = f"[ SOBORNO ACEPTADO ]\nHas evitado el combate con astucia.\n(+30 Ptos Estrategia)."
        fx_soborno = {'costo_oro': costo_soborno, 'estrategia': 30, 'texto_custom': txt_res, 'tipo_enemigo': 0}
    else:
        txt_res = f"[ ¡TRAICION! ]\nFuiste ingenuo al confiar en el monstruo.\n(-10 Ptos Estrategia)."
        fx_soborno = {'costo_oro': costo_soborno, 'vida': -dano, 'estrategia': -10, 'texto_custom': txt_res, 'tipo_enemigo': 0}

    nodo_resultado_soborno = NodoDecision(txt_res, es_hoja=False, efecto=fx_soborno)
    if dado_destino > 0.7:
        nodo_resultado_soborno.agregar_opciones(NodoDecision("Continuar a salvo", es_hoja=True, efecto={'mover_a_nodo': id_siguiete}))
    else:
        nodo_resultado_soborno.agregar_opciones(NodoDecision("Escapar malherido", es_hoja=True, efecto={'mover_a_nodo': id_siguiete}))

    nodo_atacar = NodoDecision("Atacar", es_hoja=False, efecto=fx_win_normal)
    nodo_atacar.agregar_opciones(NodoDecision("Continuar", es_hoja=True, efecto={'mover_a_nodo': id_siguiete}))

    nodo_huir = NodoDecision("Huir", es_hoja=False, efecto=fx_huir)
    nodo_huir.agregar_opciones(NodoDecision("Avanzar herido", es_hoja=True, efecto={'mover_a_nodo': id_siguiete}))

    raiz.agregar_opciones(nodo_atacar)
    if id_enemigo != 5:
        btn_sobornar = NodoDecision(f"Sobornar ({costo_soborno} Oro)", es_hoja=False, efecto=nodo_resultado_soborno.efecto)
        for op in nodo_resultado_soborno.opciones: btn_sobornar.agregar_opciones(op)
        raiz.agregar_opciones(btn_sobornar)
    raiz.agregar_opciones(nodo_huir)

    return raiz

```

```

def _arbol_sorpresa(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(texto_intro + "\nun cofre antiguo reposa en el centro.", efecto={'tipo_enemigo': 0, 'fondo_id': id_fondo})
    roll = random.random()
    if roll > 0.5:
        res = NodoDecision("Abrir con cuidado", es_hoja=False, efecto={'puntaje': 80, 'estrategia': 15, 'texto_custom': "[FORTUNA! (+80 Oro / +15 Ptos Estrategia)]"})
    else:
        res = NodoDecision("Abrir con cuidado", es_hoja=False, efecto={'vida': -20, 'estrategia': 0, 'texto_custom': "[TRAMPA! (-20 HP)]"})
    res.agregar_opciones(NodoDecision("Seguir camino", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    raiz.agregar_opciones(res, NodoDecision("Ignorar y seguir", es_hoja=True, efecto={'mover_a_nodo': id_siguiente, 'estrategia': 5}))
    return raiz

def _arbol_evento_mistico(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(texto_intro, efecto={'tipo_enemigo': 0, 'fondo_id': id_fondo})
    rezar = NodoDecision("Ofrecer plegaria", es_hoja=False, efecto={'vida': 20, 'estrategia': 10, 'texto_custom': "Sientes paz. (+20 HP / +10 Ptos Estrategia)"})
    rezar.agregar_opciones(NodoDecision("Avanzar", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    robar = NodoDecision("Profanar altar", es_hoja=False, efecto={'vida': -20, 'puntaje': 100, 'estrategia': 5, 'texto_custom': "El oro quema. (-20 HP / +100 Oro / +5 Ptos Estrategia)"})
    robar.agregar_opciones(NodoDecision("Avanzar", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    raiz.agregar_opciones(rezar, robar)
    return raiz

def _arbol_evento_trampa(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(texto_intro, efecto={'tipo_enemigo': 0, 'fondo_id': id_fondo})
    saltar = NodoDecision("Intentar saltar", es_hoja=False, efecto={'estrategia': 10, 'texto_custom': "Salto perfecto. (+10 Ptos Estrategia)"})
    saltar.agregar_opciones(NodoDecision("Seguir", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    correr = NodoDecision("Correr rapido", es_hoja=False, efecto={'vida': -15, 'estrategia': -5, 'texto_custom': "Te cortaste. (-15 HP / -5 Ptos Estrategia)"})
    correr.agregar_opciones(NodoDecision("Seguir", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    raiz.agregar_opciones(saltar, correr)
    return raiz

def _arbol_hada(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(f"{texto_intro}\nuna pequeña luz revolotea, un Hada te observa con curiosidad.", efecto={'tipo_enemigo': 8, 'fondo_id': id_fondo})
    aceptar = NodoDecision("Extender la mano", es_hoja=False, efecto={'pocion': 2, 'estrategia': 10, 'texto_custom': "El hada te bendice. (+2 Pociones / +10 Ptos Estrategia)"})
    aceptar.agregar_opciones(NodoDecision("Agradecer y partir", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))
    ignorar = NodoDecision("Ahuyentar", es_hoja=True, efecto={'mover_a_nodo': id_siguiente, 'estrategia': 0})
    raiz.agregar_opciones(aceptar, ignorar)
    return raiz

def _arbol_mercader(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(f"{texto_intro}\nEl Mercader te mira con ojos inyectados en sangre.\n'Oro... o Vida. Todo tiene un precio aqui'.", efecto={'tipo_enemigo': 9, 'fondo_id': id_fondo})

    btn_pocion = NodoDecision("Comprar Pocion (50 Oro)", es_hoja=False,
        efecto={'costo_oro': 50, 'pocion': 1, 'estrategia': 10, 'texto_custom': "Buena compra. (+1 Pocion / +10 Ptos Estrategia)", 'tipo_enemigo': 9})
    btn_pocion.agregar_opciones(NodoDecision("Despedirse", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    roll_rob = random.random()
    if roll_rob > 0.5:
        txt_rob = "¡WOW JAJAJ! Le has quitado la bolsa.\n(+100 Oro / +20 Ptos Estrategia)"
        fx_rob = {'puntaje': 100, 'estrategia': 20, 'texto_custom': txt_rob, 'tipo_enemigo': 9}
    else:
        txt_rob = "¡TE ATRAPO! Te rompe un dedo.\n(-30 HP / -15 Ptos Estrategia)"
        fx_rob = {'vida': -30, 'estrategia': -15, 'texto_custom': txt_rob, 'tipo_enemigo': 9}

    nodo_res_rob = NodoDecision(txt_rob, es_hoja=False, efecto=fx_rob)
    nodo_res_rob.agregar_opciones(NodoDecision("Huir indignado", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    btn_rob = NodoDecision("ROBAR AL MERCADER (50% Riesgo)", es_hoja=False, efecto=nodo_res_rob.efecto)
    for op in nodo_res_rob.opciones: btn_rob.agregar_opciones(op)

    roll_caja = random.random()
    if roll_caja > 0.5:
        txt_caja = "¡JOYA BRILLANTE!\n(+150 Oro / +20 Ptos Estrategia)"
        fx_caja = {'costo_oro': 30, 'puntaje': 150, 'estrategia': 20, 'texto_custom': txt_caja, 'tipo_enemigo': 9}
    else:
        txt_caja = "¡BOMBA DE BROMA!\n(-15 HP / -5 Ptos Estrategia)"
        fx_caja = {'costo_oro': 30, 'vida': -15, 'estrategia': -5, 'texto_custom': txt_caja, 'tipo_enemigo': 9}

    nodo_res_caja = NodoDecision(txt_caja, es_hoja=False, efecto=fx_caja)
    nodo_res_caja.agregar_opciones(NodoDecision("Cerrar caja", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    btn_caja = NodoDecision("CAJA MISTERIOSA (30 Oro)", es_hoja=False, efecto=nodo_res_caja.efecto)
    for op in nodo_res_caja.opciones: btn_caja.agregar_opciones(op)

    btn_sangre = NodoDecision("VENDER VITALIDAD (+50 Oro / -25 HP)", es_hoja=False,
        efecto={'vida': -25, 'puntaje': 50, 'estrategia': 10, 'texto_custom': "Medida desesperada.\n(+50 Oro / -25 HP / -10 Ptos Estrategia)", 'tipo_enemigo': 9})
    btn_sangre.agregar_opciones(NodoDecision("Despedirse debil", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    btnirse = NodoDecision("No comprar nada", es_hoja=True, efecto={'mover_a_nodo': id_siguiente, 'estrategia': 0})

    raiz.agregar_opciones(btn_pocion, btn_rob, btn_caja, btn_sangre, btnirse)
    return raiz

def _arbol_portal_caos(self, id_siguiente, id_fondo, texto_intro):
    raiz = NodoDecision(f"{texto_intro}\nun vortice inestable palpitá frente a ti.", efecto={'tipo_enemigo': 10, 'fondo_id': id_fondo})
    roll = random.random()

    if roll > 0.5:
        efecto_cruzar = {'vida': 100, 'estrategia': 5, 'texto_custom': "Curación total. (+5 Ptos Estrategia)"}
    else:
        efecto_cruzar = {'vida': -40, 'estrategia': 5, 'texto_custom': "Dolor agónico. (-40 HP / +5 Ptos Estrategia)"}

    entrar = NodoDecision("Cruzar el umbral", es_hoja=False, efecto=efecto_cruzar)
    entrar.agregar_opciones(NodoDecision("Recuperarse y avanzar", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

    roll_sorpresa = random.random()
    if roll_sorpresa > 0.5:
        efecto_sorpresa = {'fuerza': 15, 'estrategia': 15, 'texto_custom': "El vacío te otorga poder. (+15 Fuerza / +15 Ptos Estrategia)"}
    else:
        efecto_sorpresa = {'vida': -20, 'puntaje': -50, 'estrategia': -10, 'texto_custom': "La locura te consume. (-20 HP / -10 Ptos Estrategia)"}

    btn_sorpresa = NodoDecision("MIRAR AL VACIO (Sorpresa)", es_hoja=False, efecto=efecto_sorpresa)
    btn_sorpresa.agregar_opciones(NodoDecision("Alejarse aturdido", es_hoja=True, efecto={'mover_a_nodo': id_siguiente}))

```

ANEXO V. Script de python main (main.py)

```
import time
import os
import json
from game_logic import DungeonLogic
from data_manager import DataManager

def main():
    directorio_script = os.path.dirname(os.path.abspath(__file__))
    #carpeta_raiz = os.path.dirname(directorio_script)
    #ruta_shared = os.path.join(carpeta_raiz, "Shared_Data")

    # ruta_shared = os.path.join(os.path.dirname(os.path.abspath(__file__)), "Shared_Data")
    ruta_assets = os.path.join(directorio_script, "..")
    ruta_shared = os.path.join(ruta_assets, "Shared_Data")
    ruta_shared = os.path.normpath(ruta_shared)

    if not os.path.exists(ruta_shared):
        os.makedirs(ruta_shared)

    print(f"Python escribiendo en: {ruta_shared}")
    print("--- RPG TORNEO DUNGEON: LISTO ---")
    logic = DungeonLogic()
    logic.iniciar_juego()

    data_mgr = DataManager()
    data_mgr.ruta_estado = os.path.join(ruta_shared, "game_state.json")
    data_mgr.ruta_accion = os.path.join(ruta_shared, "player_action.json")

    sala_inicio = logic.grafo.obtener_sala(logic.jugador.nodo_actual)
    nodo_arbol_actual = sala_inicio.arbol_decision

    texto_historia_fijo = nodo_arbol_actual.texto
    texto_feedback_temporal = ""

    logic.imagen_evento_id = nodo_arbol_actual.efecto.get('tipo_enemigo', 0)
    fondo_actual = nodo_arbol_actual.efecto.get('fondo_id', 0)
```

```

ultimo_tiempo_pocion = 0
COOLDOWN_POCION = 1.5

ultimo_tiempo_accion = 0
COOLDOWN_ACCION = 0.5

veces_necio = 0
toggle_visual = False

print("Esperando a Unity...")

try:
    while True:
        # 1. RENDERIZADO
        if texto_feedback_temporal != "":
            logic.texto_evento = f"{texto_feedback_temporal}\n-----\n{texto_historia_fijo}"
        else:
            logic.texto_evento = texto_historia_fijo

        # 2. GUARDAR
        logic.fondo_visual_id = fondo_actual
        data_mgr.guardar_estado_juego(logic, nodo_arbol_actual)

        # 3. LEER ACCIÓN
        accion = data_mgr.leer_accion_jugador()

        if accion:
            with open(data_mgr.ruta_accion, 'w') as f:
                f.write("{}")

            tipo = accion.get("accion")
            tiempo_actual = time.time()

```